**DevFoundry**

# Build & Deploy Your First Website

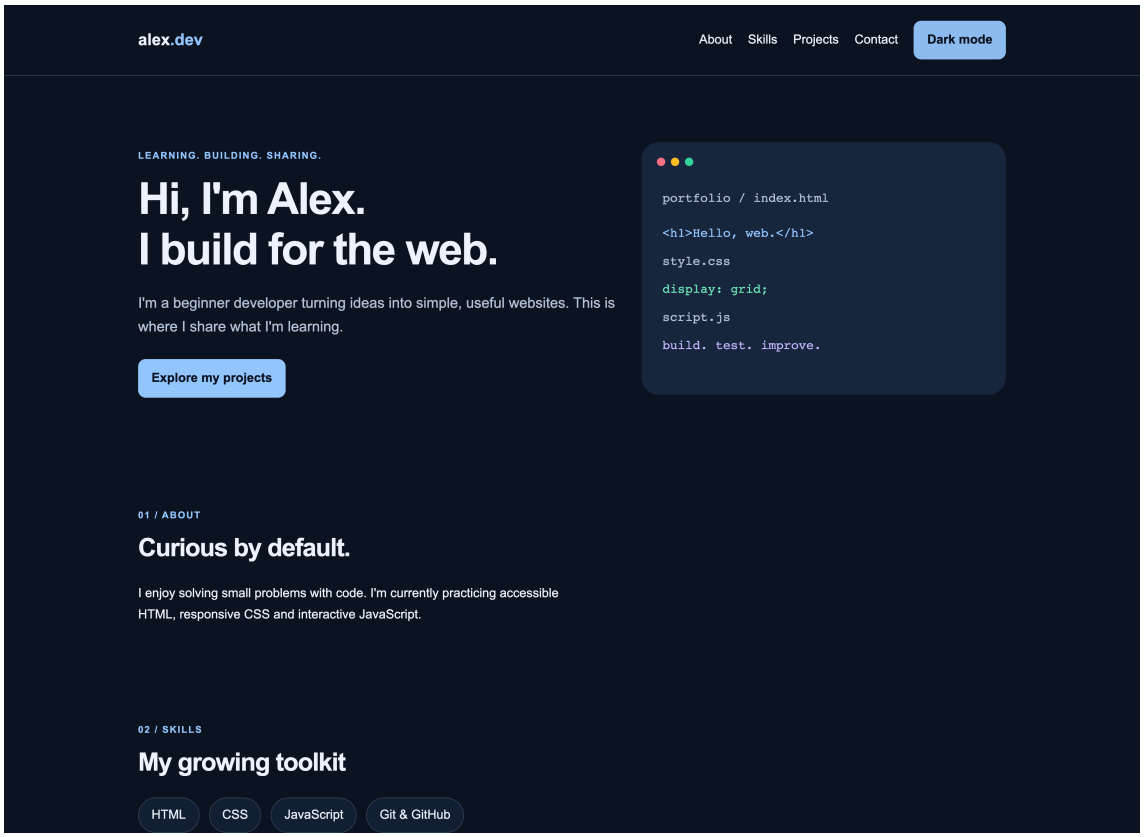
## From Zero to a Live Website with HTML, CSS & JavaScript

A complete beginner course and practical workbook

Start with an empty folder. Finish with a responsive personal portfolio that you can publish and keep improving.

HTML • CSS • JavaScript • Git • GitHub Pages

Edition 1.0 | November 2026




The actual course portfolio, shown in dark mode.

# Welcome to DevFoundry

Your task in this course is to build and publish one website. Each module adds a useful layer: content, design, responsive layout, interaction, version history and deployment. You will understand what your code does instead of only collecting snippets.

You do not need previous programming experience. You do need a computer with permission to install an editor and Git, a modern browser, an internet connection for setup and publishing, and a GitHub account. A tablet or phone alone is not the intended coding environment.

The original guided course is planned for about 2–3 hours of instruction. Reading, typing, debugging and completing the challenges will usually take longer. Plan several short sessions and work at your own pace. The final project is additional practice, not a timed exam.

## What you will be able to do

- Write a complete HTML page with meaningful headings and sections.
- Style it with reusable CSS, Flexbox and Grid.
- Adapt the layout from a 320px phone viewport to a wide desktop.
- Build a theme toggle, mobile menu and project filter with JavaScript.
- Save meaningful Git commits, publish a GitHub repository and explain your work in a README.
- Deploy a static website with GitHub Pages and make a verified live update.

## How to use this workbook

Read a lesson, type its example, then complete its task before moving on. Save the file and refresh the browser after every meaningful change. Examples labelled “experiment” are temporary practice. Examples labelled “portfolio” belong in your working project. A code block is a focused excerpt unless it explicitly says to replace the whole file.

Use one working folder named devfoundry-portfolio throughout. The ZIP provides complete snapshots for comparison at the end of each major stage. If your code breaks, compare the relevant file with the matching checkpoint rather than replacing your whole project immediately. The appendix contains the full final HTML, CSS and JavaScript, also supplied as editable files.

The reference name Alex Morgan, email address and planned project concepts are example content. Replace them before publishing. You can use honest learning projects; you do not need to invent client work or experience.

# Your course map

Module 0 Getting started 4

Module 1 HTML Build the structure 9

Module 2 CSS Make it look good 13

Module 3 Responsive design 19

Module 4 JavaScript Make it interactive 25

Module 5 Git and GitHub 30

Module 6 Deploy your website 33

Module 7 Final project 36

Module 8 Where to go next 39

Course downloads and quick references 40

CSS cheat sheet 42

Challenges and answer guide 44

Troubleshooting desk 46

Resource directory 47

Appendix A Complete portfolio HTML 48

Appendix B Complete portfolio CSS 51

Appendix C Complete portfolio JavaScript 54

## Suggested learning sessions

Session 1: set up your tools and build the HTML skeleton. Session 2: apply CSS and compare the design reference. Session 3: fix the mobile layout and test it. Session 4: build the interactions. Session 5: create your GitHub repository and deploy. Session 6: personalize, test and submit the final portfolio.

## Your build milestones

Empty folder → HTML skeleton → styled portfolio → responsive portfolio → interactive portfolio → GitHub repository → live website.

Each project includes observable pass criteria. If a checkpoint fails, use the troubleshooting section and repeat the check before adding the next layer.

# Module 0 Getting started

Goal: understand the finished website and prepare a reliable workspace. Guided time: about 10 minutes, excluding downloads and account setup.

## Lesson 0.1 What we are building

Your finished portfolio has a navigation bar, introduction or hero section, About section, skill list, three project cards, contact section and footer. Visitors should be able to understand who you are, see your work and find a way to contact you. On smaller screens the content stacks, and the navigation opens with a button. A theme control switches the color palette without changing the content.

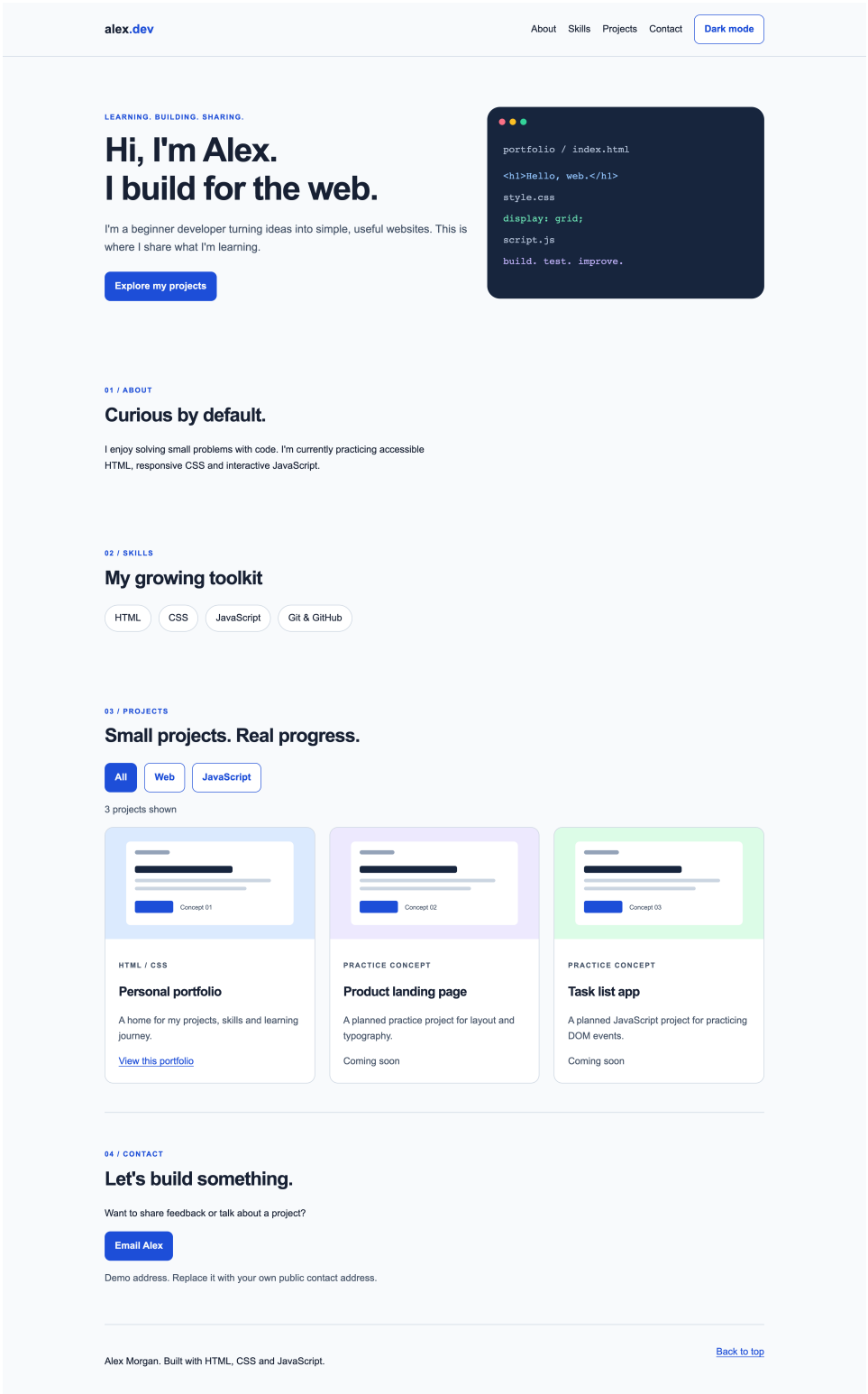



Figure 1. Full-page desktop target. All seven page regions belong to one continuous portfolio.

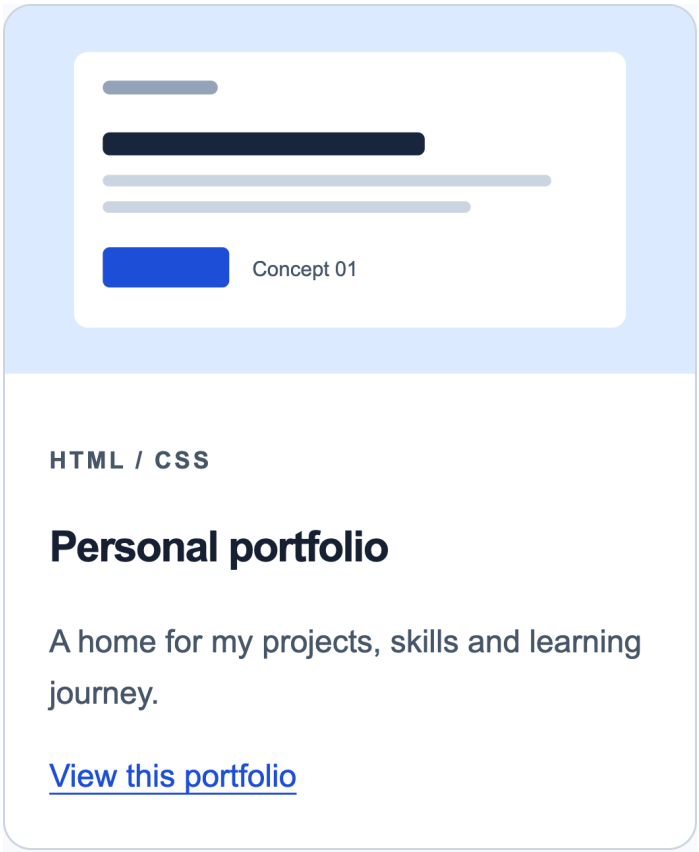
LEARNING. BUILDING. SHARING.

# Hi, I'm Alex.
# I build for the web.

I'm a beginner developer turning ideas into simple, useful websites. This is where I share what I'm learning.

Explore my projects

Detail: the introduction and primary action, enlarged for reading.




Detail: a complete project card with image, title, description and destination.

The reference is a target for hierarchy and layout, not a test of whether your name or colors match the example. Notice the clear introduction, repeated card design and generous space between sections. The project cards below the hero are the same width and use consistent image proportions.

### Do this now

Write your name, a one-sentence introduction, four skills, three project ideas and a contact method in a note. If you have only built one project, describe the others as planned practice concepts. Choose a public contact address you are comfortable sharing.

### Check your understanding

What does a portfolio help a visitor do? It should explain your focus, show evidence of your work and offer a next step. A beautiful layout with no useful content is not finished.

## Lesson 0.2 How websites work

A browser reads HTML to understand the content, applies CSS to lay it out and style it, and runs JavaScript to respond to actions. A server sends these files when someone requests a page. Hosting is the service that makes the files available. A domain is a name used to find a site; a URL includes details such as the protocol and page path.

Frontend development covers the page people see and interact with. Backend systems handle work such as accounts, databases and private server logic. Our course project is a static frontend website: it does not need a database or backend process.

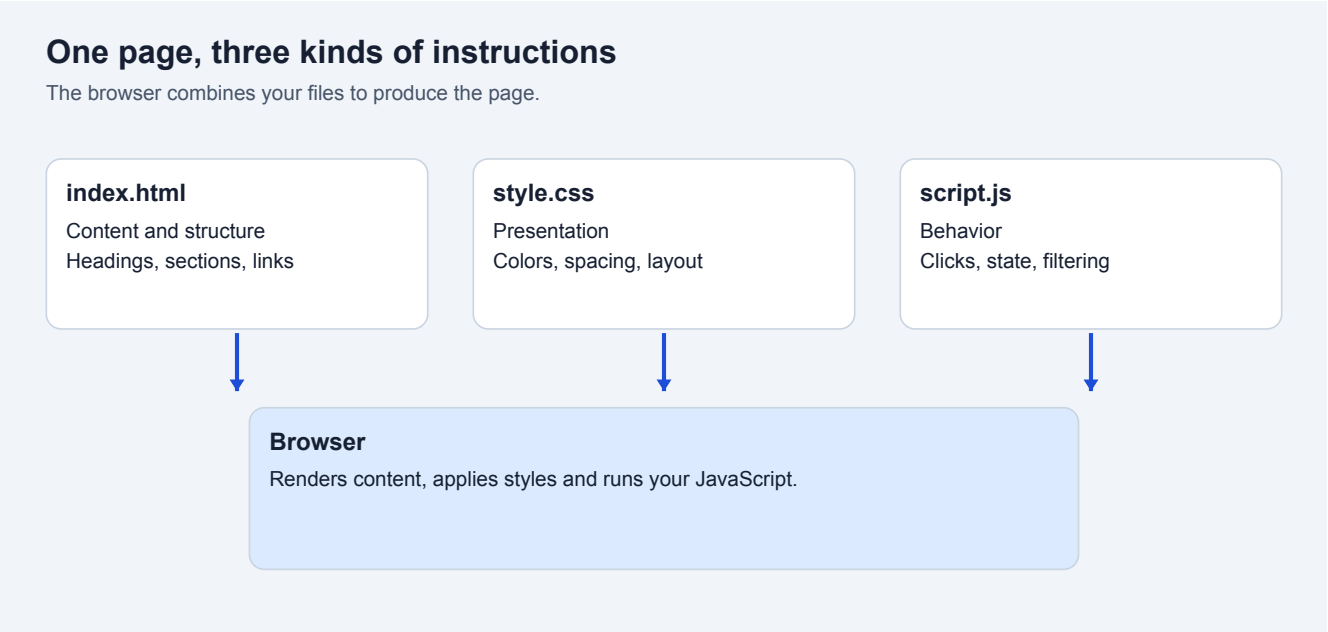



Figure 2. The same browser combines three file types with different responsibilities.

### Try it

Open a familiar website and resize the browser. Look for changes in columns, spacing and navigation. Then explain the three roles aloud: HTML describes content, CSS controls presentation, and JavaScript adds behavior. Turning JavaScript off does not remove HTML or CSS, although some sites rely heavily on scripts.

# Lesson 0.3 Set up your tools

Install Visual Studio Code from code.visualstudio.com and Git from git-scm.com. Use a current Chrome, Edge, Firefox or Safari browser. Create a GitHub account at github.com and verify your email. These services may change their screens; official reference links are in the resource directory at the end of this workbook.

1. Extract the course ZIP. Copy 01-starter-files to a location you can find, then rename the copy devfoundry-portfolio.
2. In VS Code, choose File → Open Folder and select devfoundry-portfolio. Opening the folder makes file paths and the terminal easier to manage.
3. Confirm index.html, style.css and script.js are present. The assets folder contains the course illustrations.
4. Open index.html in your browser through your file manager. It will be blank initially. You can also use a local preview extension in VS Code; no extension is required for this project.
5. Open Terminal → New Terminal in VS Code. Run the following check. The output should include a version number.

```
git --version
```

On Windows, use Git Bash if you want the terminal examples to match closely. On macOS, use the built-in terminal in VS Code. If Git is not found after installation, restart VS Code. Do not type the opening prompt character shown by your terminal, only the command itself.

### Checkpoint 1

You can edit and save index.html, reopen it in a browser, and run Git. Confirm filenames do not end in .txt. Keep the project outside another existing Git repository if possible. The course ZIP contains no .git history.

# Module 1 HTML Build the structure

Goal: create meaningful content and a complete page before styling it. Guided time: about 25 minutes. Checkpoint: 02-html-complete.

## Lesson 1.1 Your first HTML page

Replace the starter index.html with this complete first page. An element usually has an opening tag, content and a closing tag. The html element contains the head and body. The head holds document information; the body holds visible page content. Indentation shows nesting and makes mistakes easier to spot.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1">
  <title>My Portfolio</title>
</head>
<body>
  <h1>Hello!</h1>
  <p>I am building my first website.</p>
</body>
</html>
```

The doctype requests modern HTML behavior. lang describes the document language. UTF-8 supports ordinary text characters. The viewport declaration lets the page use the device's actual viewport width, which matters for responsive CSS later. The title appears in the browser tab; h1 appears on the page.

### Build and check

Save, refresh, and confirm the heading and paragraph appear. Change the title and refresh again; the browser tab should change. If the code itself appears as text, check the .html extension and that you opened the saved file. Closing tags use a forward slash, such as </p>.

## Lesson 1.2 Text and content

Use headings to describe a hierarchy, not to choose a text size. Give this one-page portfolio one main h1 and an h2 for each major section. Use h3 for each project title. Paragraphs are p elements. strong marks importance, and em marks emphasis. br creates a line break inside text, not space between sections; CSS will handle spacing.

Portfolio example: insert this section after the introduction inside body. Later it will move into main.

```
<section id="about">
  <h2>About me</h2>
  <p>I'm Alex, a <strong>beginner developer</strong>.</p>
  <p>I enjoy building useful websites and
    <em>learning by doing</em>.</p>
</section>
```

### Mini project About Me

Replace the example with two short paragraphs about yourself. Include what you are learning and what you enjoy making. Use strong or em only when the meaning justifies it. Your section passes when the browser displays an h2 followed by readable paragraphs and your page still has a clear main h1.

## Lesson 1.3 Links and images

An anchor, a, navigates to another location. Its href gives the destination. A fragment link such as #about moves to the element whose id is about. IDs must be unique within the page. An image uses src for its file path and alt for a text alternative. img is a void element: it has no closing </img> tag.

```
<a href="https://github.com/">Visit GitHub</a>
<a href="#about">About me</a>
<img src="assets/workspace.svg"
  width="520" height="360"
  alt="Illustrated code editor with HTML, CSS and JavaScript files">
```

Paths are relative to the HTML file. assets/workspace.svg means “look inside the assets folder beside this page.” Use forward slashes and match capitalization exactly. Meaningful images need useful alt text; purely decorative images can use alt="". Avoid repeating a caption word for word in the alt text.

### Mini challenge

Add the bundled workspace image, a link to GitHub and a fragment link to your About section. Click both links. Deliberately rename the image path, observe the failure, then fix it. Do not use a path from your own computer such as C:\\Users\\Alex: other visitors cannot access it.

## Lesson 1.4 Divs and semantic HTML

Semantic elements describe a region's purpose. header introduces a page or region, nav groups primary navigation, main contains the page's main content, section groups a named topic, article holds a self-contained item, and footer closes a page or region. A div is a neutral container useful for layout when no more meaningful element fits.

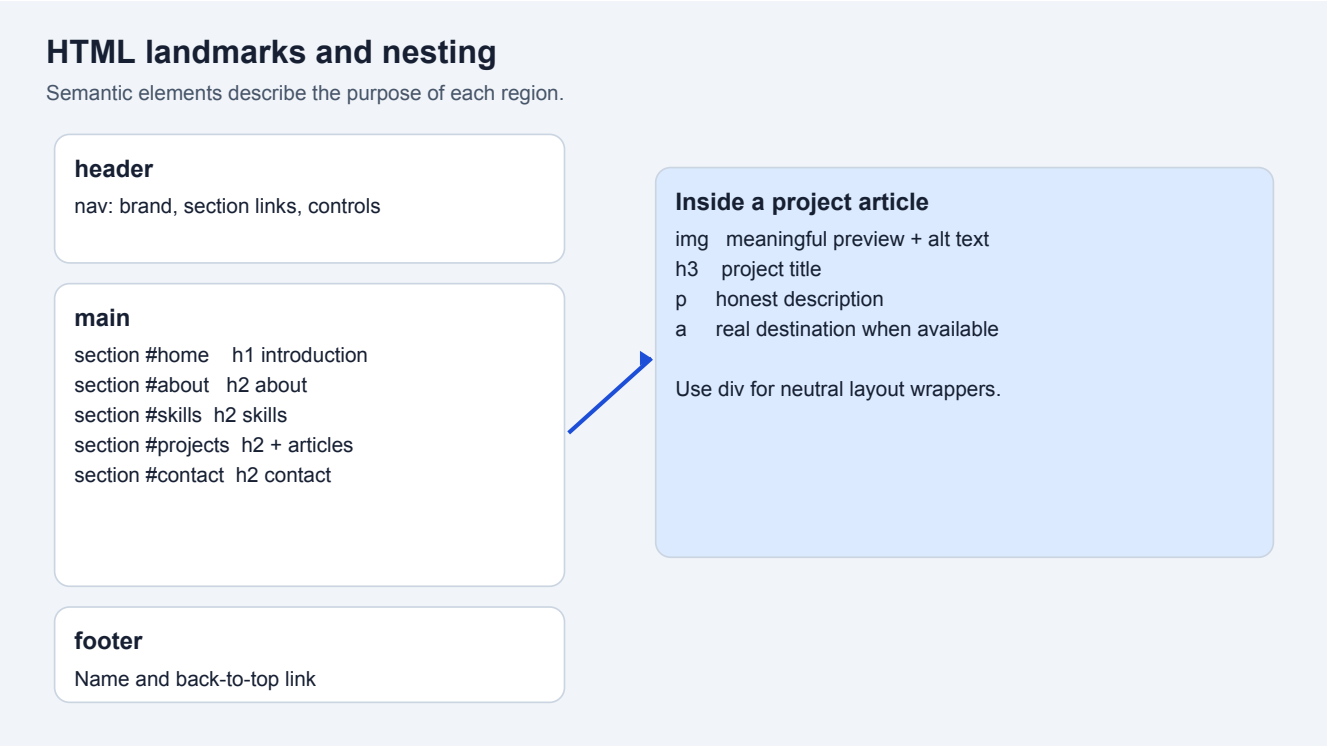



Figure 3. Landmarks and nested project content. Tags communicate purpose before styling.

Use one main landmark. Put navigation in header, the five main content sections in main, and the footer after main. A section usually needs a heading. Our project cards use article because each is an independent project summary. The wrapper around links can remain a div; it does not need an invented semantic meaning.

### Practice and check

Reorganize your existing content into header, main and footer. Keep all visible text inside body. Your page may look identical: semantic structure improves meaning without automatically creating a design. Confirm that #about still finds its section after moving it.

## Lesson 1.5 Build the portfolio structure

Expand main into the full page shown below. This is the structural plan; the complete content is in Appendix A and 02-html-complete/index.html. Copy the full checkpoint HTML if you need a comparison, then replace the example text with your own. Do not connect CSS or JavaScript yet.

```
<header><!-- navigation --></header>
<main id="main">
  <section id="home"><h1>Your introduction</h1></section>
  <section id="about"><h2>About</h2></section>
  <section id="skills"><h2>Skills</h2></section>
  <section id="projects"><h2>Projects</h2></section>
  <section id="contact"><h2>Contact</h2></section>
</main>
<footer><!-- name and back-to-top link --></footer>
```

Use an unordered list, ul with li items, for skills. Create three article elements for project cards, each with an image, h3 and description. A finished project can have a real destination link. Planned

projects should show “Coming soon” as text rather than a misleading dead link. The complete reference also includes hidden controls for Module 4; hidden keeps these inactive buttons out of the page until their behavior exists.

### Project 1 Portfolio skeleton

Deliverable: one complete, unstyled HTML page. Your name and introduction appear first, followed by About, Skills, Projects and Contact. Include a footer. At least three cards must exist, even if two are clearly labelled planned concepts. Use a mailto link for contact; it opens the visitor's configured mail app and does not send a message by itself.

Pass when every navigation link reaches its matching unique ID, each image loads or has a meaningful alternative, the heading order is sensible, and no content is outside body. Compare against 02-html-complete. A plain page at this stage is expected; the next module gives it a design.

# Module 2 CSS Make it look good

Goal: create a consistent desktop design. Guided time: about 35 minutes. Checkpoint: 03-css-complete. This checkpoint deliberately uses a fixed desktop width so you can improve it in Module 3.

## Lesson 2.1 Connect CSS

Inside head, add the stylesheet link. Then put the body rule in style.css. Save both files and refresh. The change proves that the stylesheet is connected.

```
<link rel="stylesheet" href="style.css">
```

```
body {
  background: #f8fafc;
  color: #172033;
  font-family: Arial, Helvetica, sans-serif;
}
```

A selector chooses elements; declarations inside braces assign values. body selects an element type, .hero selects a class, and #projects selects an ID. Multiple elements can share a class. An ID identifies one element. Put class="hero" on a section to make .hero match it.

When two rules target the same property, the cascade decides which wins. For these examples, a more specific selector usually wins, and equally specific rules later in the stylesheet win over earlier ones. Use classes for most styling and inspect unexpected results in the browser's Styles panel instead of scattering !important declarations.

### Build and check

Connect the stylesheet and change your h1 color temporarily. Restore it afterward. If nothing changes, check style.css spelling, save the file, and inspect the browser's Network or Console panels for a failed request. The Console is in developer tools, available through your browser's menu or by inspecting the page.

## Lesson 2.2 Colors fonts and variables

Reusable design values keep a website consistent. Custom properties, often called CSS variables, begin with two hyphens. :root is the top-level HTML element, so variables declared there are available throughout the document. var() reads a value.

```
:root {
  --bg: #f8fafc;
  --surface: #ffffff;
  --text: #172033;
  --muted: #475569;
  --accent: #1d4ed8;
  --line: #cbd5e1;
}
body {
  background: var(--bg);
  color: var(--text);
  font-family: Arial, Helvetica, sans-serif;
  font-size: 1rem;
  line-height: 1.65;
}
```

The background is the page canvas, surface is a card background, text is the main copy, muted is secondary copy, accent highlights links and buttons, and line separates regions. Choose readable contrast rather than assuming every bright color works as text. Browser accessibility tools can help measure contrast.

### Practice and check

Apply the variables to body, links and project cards. Change only --accent and verify that all components using it update together. Keep comfortable body line spacing and a limited number of text sizes. The full palette and type scale appear in Appendix B.

## Lesson 2.3 The CSS box model

Every element occupies a box. Content is the text or image. Padding creates space inside its border. Margin creates space outside the border. Border-radius rounds the corners but does not add space. A box's visible dimensions depend on box-sizing.

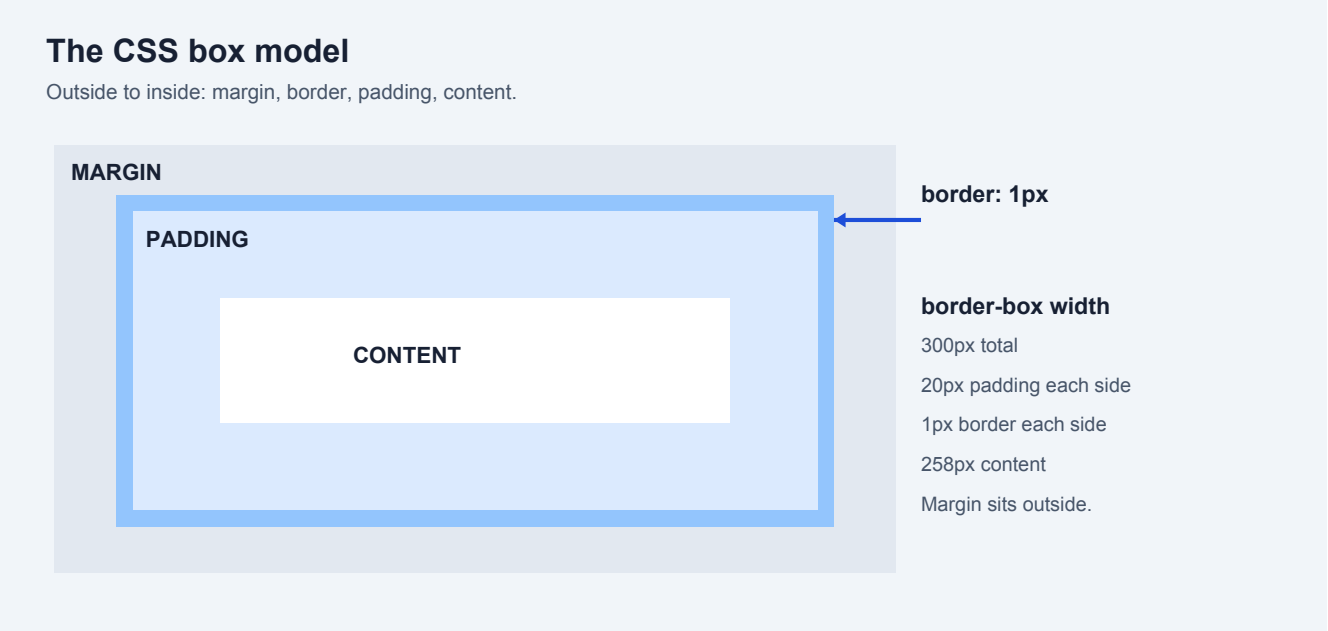



Figure 4. With border-box, declared width includes content, padding and border, while margin stays outside.

```
* { box-sizing: border-box; }
.project-card {
  width: 300px;
  padding: 20px;
  border: 1px solid var(--line);
  border-radius: 12px;
  margin-bottom: 20px;
}
```

With border-box, the 300px width includes padding and border: the content is 258px wide. With the default content-box, the border box would be 342px wide. Margins sit outside both measurements. In the finished Grid layout, remove this fixed card width so the columns control it; the final card uses padding on .card-body instead.

### Mini challenge Project card

Temporarily style one article with the rule above. In developer tools, inspect its box model and identify the four layers. Increase padding and describe what changes. Remove the temporary width and padding before applying the final .project-card and .card-body rules from Appendix B.

## Lesson 2.4 Flexbox

Flexbox arranges a container's direct children along a main axis. With the default flex-direction: row, the main axis runs horizontally and the cross axis vertically. Change to column and the axes rotate with it. justify-content aligns along the main axis; align-items aligns along the cross axis. gap separates children without adding an outer margin.

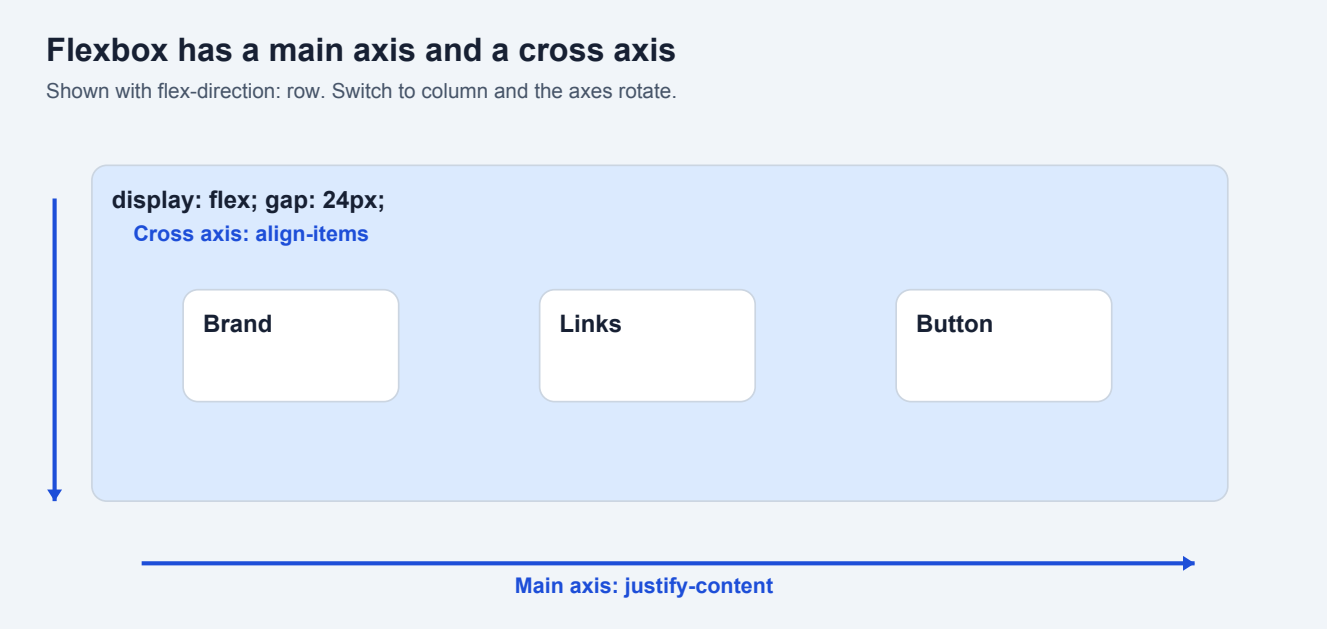



Figure 5. Main-axis and cross-axis alignment for a row-based Flexbox container.

```
.nav {
  display: flex;
  align-items: center;
  gap: 1.25rem;
}
.brand { margin-right: auto; }
.nav-links { display: flex; gap: 1.25rem; }
.hero { display: flex; align-items: center; gap: 3rem; }
.hero > div { flex: 1; }
.hero-art { width: 42%; height: auto; }
```

The brand's automatic right margin absorbs spare horizontal space. The hero's text wrapper grows to use available space; the image occupies part of the row. These rules assume the class names from Appendix A. Only direct children become flex items; a nested paragraph is not automatically a flex item of .hero.

### Build and check

Apply these rules to your navigation and hero. Add display: flex, flex-wrap: wrap and gap to .skills so badges can wrap. Try flex-direction: column temporarily and observe why axis language is more accurate than “horizontal alignment.” Restore the desktop hero row for now.

## Lesson 2.5 CSS Grid

Grid controls rows and columns together. Use it for repeated project cards. Each article is a direct child of .projects-grid. A fraction, fr, shares available track space after gaps. minmax(0, 1fr) lets the columns shrink without a long child forcing its track wider than intended.

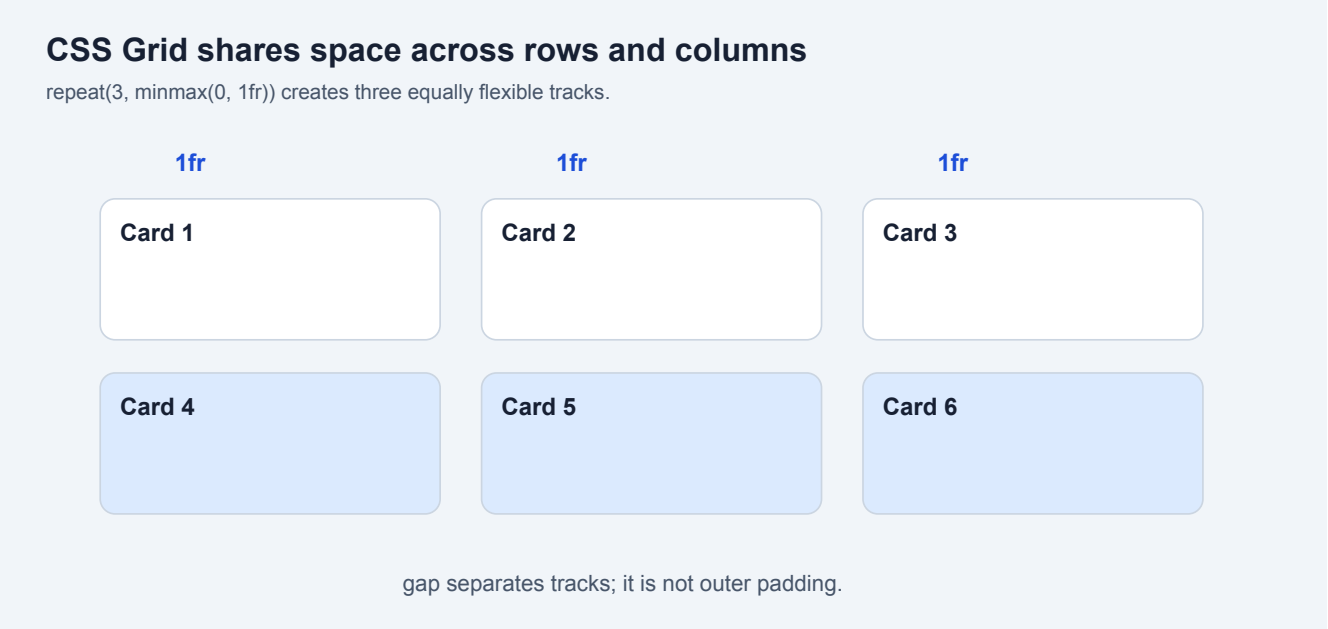



Figure 6. Grid tracks share available space. A fourth card flows into a new row.

```
.projects-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 1.5rem;
}
.project-card {
  overflow: hidden;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: .9rem;
}
.card-body { padding: 1.4rem; }
.project-card img { width: 100%; height: auto; }
```

**Build and check**

Put the three project articles inside a div with class projects-grid. Apply the rule and confirm three equal columns at the desktop reference width. Change repeat(3, ...) to repeat(2, ...) as an experiment: the third card should start another row. Restore three columns. These desktop columns will be too tight on a phone; that is the specific problem Module 3 solves.

## Lesson 2.6 Hover effects and transitions

A hover state gives mouse users feedback. A visible focus state serves keyboard users. A transition smooths a change between values; transform moves the card visually without rearranging neighboring cards. Keep the motion small and purposeful.

```
.project-card { transition: transform .2s ease; }
.project-card:hover,
.project-card:focus-within { transform: translateY(-4px); }
:focus-visible {
  outline: 3px solid var(--accent);
  outline-offset: 4px;
}
@media (prefers-reduced-motion: reduce) {
  .project-card { transition: none; }
  .project-card:hover,
  .project-card:focus-within { transform: none; }
}
```

**Build and check**

Hover a card, then use Tab to focus its link. Both interactions should provide feedback. Do not remove focus outlines. A touch device may not have hover, so no essential information should require it. Reduced-motion preferences turn this movement off in the final stylesheet.

## Lesson 2.7 Complete the website design

Use the complete base CSS in Appendix B, stopping before the “3.2 Mobile first overrides and breakpoints” comment. This is the same content as 03-css-complete/style.css. Read each group before adding it: theme and type, shared elements, Flexbox, Grid and feedback. Replace temporary practice rules that target the same classes.

The base design uses an 1100px .container centered with margin-inline: auto. Sections share padding. Buttons and links repeat the accent color. Project images use the same aspect ratio, while card text is allowed to grow naturally. Heights are not fixed around paragraphs, so longer text does not get clipped.

### Project 2 Portfolio redesign

Deliverable: a styled desktop portfolio. Test in a browser wider than 1200px. Check the full page against the reference: readable type, a strong hero, equal card columns, useful contrast, consistent borders and rounded corners, and obvious links. Confirm the skip link appears when focused with the keyboard.

Pass when the design feels consistent from the header to the footer, images are not distorted, and keyboard focus is visible. Do not hide horizontal overflow as a shortcut. You will replace the rigid container and change the layout in Module 3.

# Module 3 Responsive design

Goal: make your design adapt smoothly to available space. Guided time: about 20 minutes. Checkpoint: 04-responsive-complete.

## Lesson 3.1 Why websites break on phones

A desktop layout breaks when its assumptions no longer fit. A fixed 1100px container overflows a 390px viewport. Three narrow cards make text hard to read. A long navigation row can push controls off screen. Responsive design changes those constraints instead of simply shrinking the whole page.

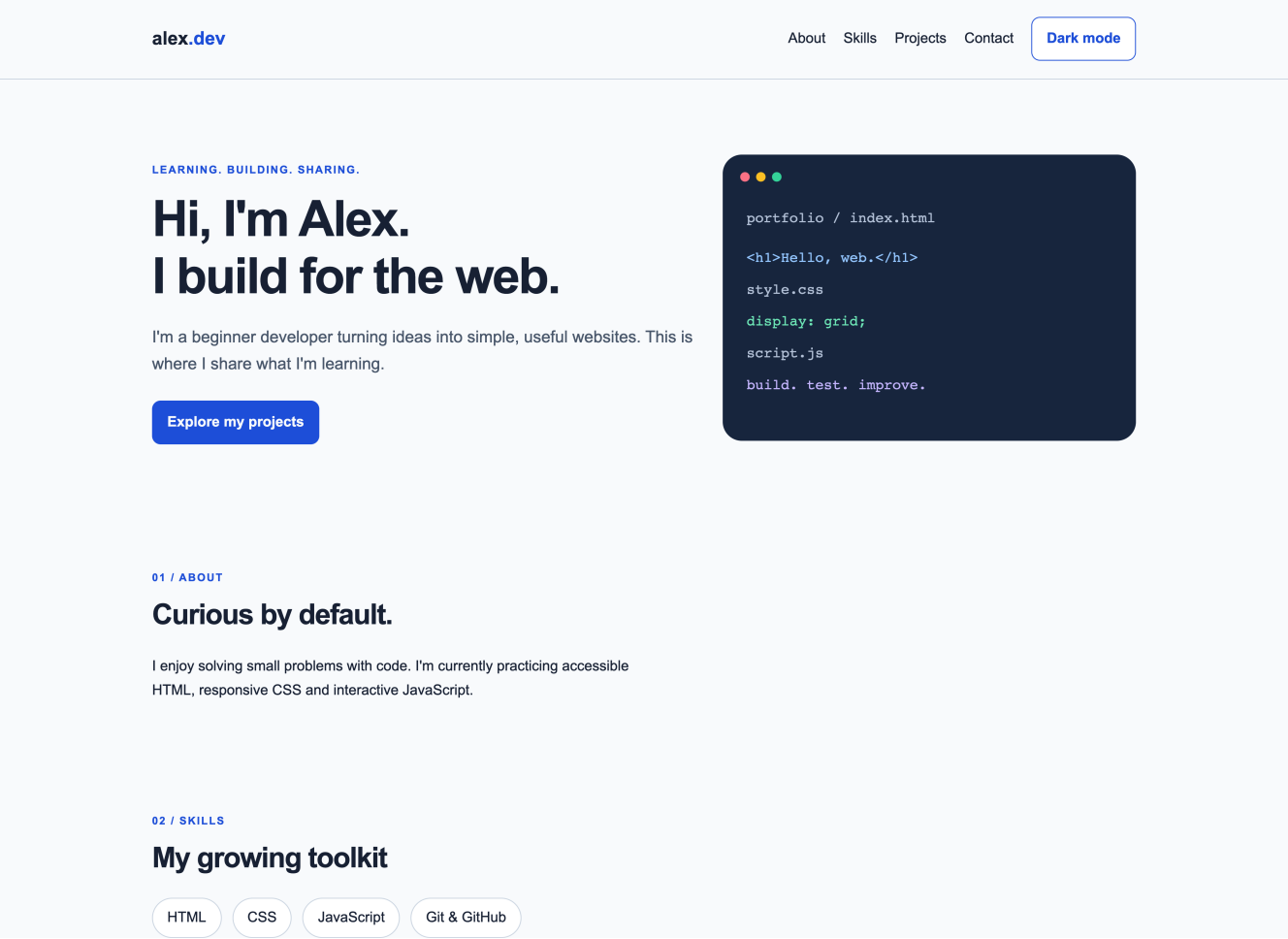
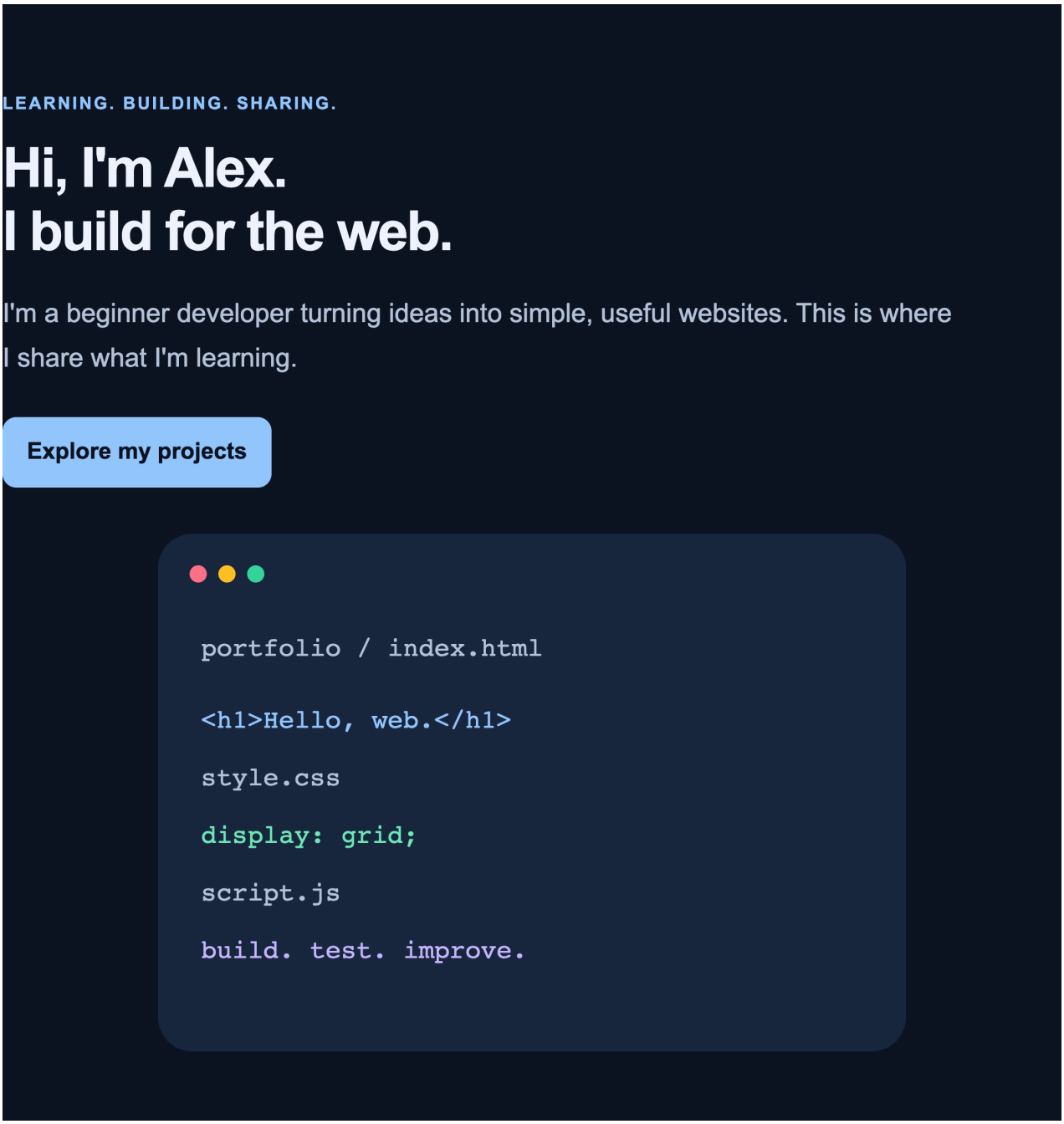
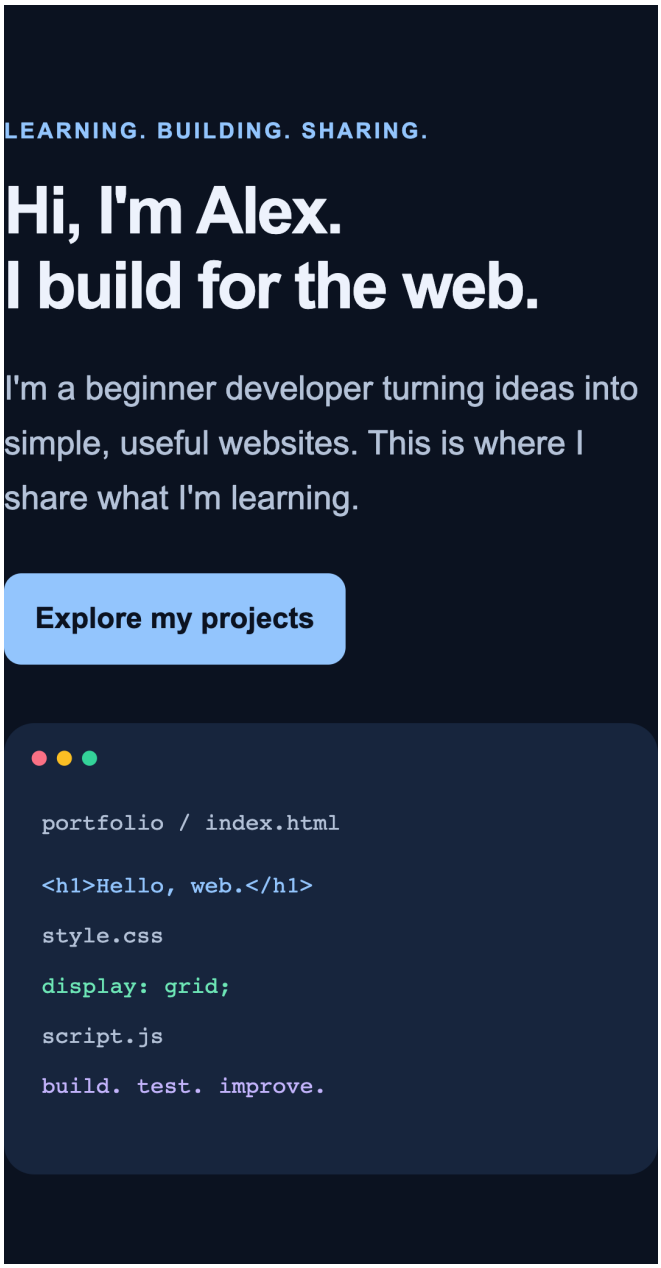



Figure 7. Matching screenshots at desktop, tablet and phone widths. Layout changes with space, not device names. Desktop overview.




Tablet at 768px. Enlarged crop of the top of the same page.




Phone at 390px. Enlarged crop of the top of the same page.

Open developer tools, enable the responsive device toolbar and reduce the viewport width. Find the first element that no longer fits. Inspect its computed width and its parent before changing CSS. You can also resize the normal browser window. A preset device frame is useful, but it does not test every intermediate size.

### Try it and check

At 390px, identify one source of horizontal scrolling in the desktop checkpoint. It should be the fixed .container. Write down the cause before applying the next lesson's fix. Do not use overflow-x: hidden on body to hide the problem: that can conceal inaccessible content.

# Lesson 3.2 Media queries

Start with the smallest layout, then add columns when there is room. Append these rules after your desktop base CSS, or use the whole 04-responsive-complete/style.css. The later equal-specificity rules override the earlier desktop values.

```
.container { width: min(1100px, calc(100% - 2rem)); }
img { max-width: 100%; height: auto; }
.projects-grid { grid-template-columns: 1fr; }
@media (min-width: 640px) {
  .projects-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}
@media (min-width: 960px) {
  .projects-grid {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }
}
```

The container keeps 1rem of space at each side until it reaches its maximum width. A media query applies when its condition is true; it does not detect a specific device model. Our breakpoints are design choices: one column below 640px, two from 640px to 959px, and three at 960px and above. Change them later if your content needs different widths.

### Check

Test at 639, 640, 959 and 960 CSS pixels. The column changes should happen at the declared boundaries, with no missing cards. Use CSS pixels in the device toolbar; a phone's physical screen pixel count is a different measurement.

# Lesson 3.3 Responsive units

px is a CSS pixel and works well for borders and breakpoints. rem is relative to the root font size; 1rem is commonly 16px at the browser default, but visitors can change it. Percentages depend on the property and containing context. vw is one percent of viewport width; vh is one percent of viewport height. Avoid assuming a full-height mobile hero will always fit browser controls.

```
h1 { font-size: clamp(2.25rem, 5vw, 3.5rem); }
.hero { flex-direction: column; gap: 2rem; }
.hero-art { width: min(100%, 520px); }
@media (min-width: 960px) {
  .hero { flex-direction: row; align-items: center; }
  .hero-art { width: 42%; }
}
```

clamp() chooses a flexible value between a minimum and maximum. Here it prevents the headline becoming tiny or enormous. width: min(...) chooses the smaller value, limiting the image while allowing it to shrink. The appendix includes all responsive overrides, including the footer layout.

### Practice and check

Increase browser text size or zoom to 200%. Make sure headings still wrap, body text remains readable and content is not clipped by fixed heights. Prefer flexible sizing and natural document flow over trying to position each item with exact coordinates.

## Lesson 3.4 Mobile navigation

Before JavaScript, keep all navigation links visible. Flex-wrap allows the desktop navigation to wrap. On small screens, the links become a column. The menu button stays hidden until its click behavior exists in Module 4. This keeps the page usable while you are still building it.

```
.nav { flex-wrap: wrap; }
@media (max-width: 639px) {
  .nav-links {
    width: 100%;
    flex-direction: column;
    order: 3;
  }
  .nav-links a { padding-block: .5rem; }
  .js .nav-links { display: none; }
  .js .nav-links.is-open { display: flex; }
}
```

The .js class is added to html only by the completed menu script. Do not manually add it yet. On a large screen, the small-screen hiding rules do not apply. The finished menu is an ordinary button controlling ordinary navigation links; it does not need application-style menu roles.

### Check

At 320px, every link is visible, readable and easy to activate. At desktop width, links form a row. Later, the button's aria-expanded state must match whether those links are open on small screens.

## Lesson 3.5 Test your website

Testing is a repeatable activity, not a final glance. Use the following matrix and record issues in challenges/test-log.md. Start with the smallest screen and move upward; also drag the window between sizes.

| Test | Expected result |
| --- | --- |
| 320px phone | One column, readable heading, no horizontal page scrolling |
| 390px phone | Comfortable spacing and usable navigation |
| 768px tablet | Two project columns and a stacked hero |
| 1440px desktop | Three project columns, hero text beside image |
| 200% zoom | Text and controls remain available without clipping |

| Test | Expected result |
| --- | --- |
| Keyboard only | Every link is reachable and focus is visible |

### Project 3 Responsive portfolio

Deliverable: a single page usable from 320px through desktop. Append the complete responsive block in Appendix B and compare with 04-responsive-complete. There should be no fixed card widths left from the box-model experiment.

Pass when images shrink proportionally, the hero and footer stack on small screens, projects use the expected column counts, navigation stays accessible and there is no horizontal overflow. Include one screenshot from each target layout in your own test notes. A real phone test is useful when available; the browser preview is still a required baseline.

# Module 4 JavaScript Make it interactive

Goal: make controls respond predictably while keeping the content usable. Guided time: about 25 minutes plus the Project 4 lab. Checkpoint: 05-javascript-complete.

## Lesson 4.1 What JavaScript does

HTML provides the controls and content. CSS defines their visible states. JavaScript changes state when something happens. For the theme toggle, a click changes a class on body, and the CSS variables change the palette. For the mobile menu, a click changes whether the navigation links are open. For filtering, a click changes which project cards are hidden.

Keep links for navigation and buttons for actions. A div with a click handler does not automatically support a keyboard the way a native button does. Our page's content and links remain usable if JavaScript is unavailable; the enhanced controls are revealed only after their setup code runs.

### Plan before coding

For each feature, describe the action, the state and the visible result. Example: “Click Dark mode → dark is true → body has class dark and the button reports pressed.” A precise sentence makes the code easier to test.

## Lesson 4.2 Your first JavaScript

Add this script element inside head, after the stylesheet link. defer downloads the script without blocking HTML parsing and runs it after the document has been parsed, so the elements are available to select.

```
<script src="script.js" defer></script>
```

Replace the temporary comment in script.js with this experiment:

```
console.log("Hello DevFoundry!");
```

Save and refresh. Open the browser's developer console and find the message. If it does not appear, confirm script.js is spelled correctly and check for a failed file request. A syntax error can stop the script from running; read the first error and its line number before adding more code.

### Check

Change the message, save and refresh again. You should see the new text. Remove this experiment before adding the completed project script. A console message proves execution, but it is not a visible user interaction.

## Lesson 4.3 Variables and functions

const creates a binding you will not reassign. let allows reassignment. Strings hold text, numbers support arithmetic, and booleans represent true or false. A function groups instructions and can accept inputs, called parameters. Defining a function does not run it; call it with parentheses.

```
const name = "Alex";
let projectsBuilt = 1;
function sayHello(person) {
  return `Hello ${person}`;
}
console.log(sayHello(name));
projectsBuilt = projectsBuilt + 1;
console.log(projectsBuilt);
```

Template literals use backticks and ${...} to insert a value into text. return sends a value back to the caller. The function parameter person is a local name for the supplied input. In our theme function, a boolean will decide whether the dark class should exist.

### Mini challenge

Write a function describeProject(title) that returns “I built ” followed by the title. Call it with your portfolio's title and log the result. Expected answer: return `I built ${title}`. Replace this experiment with your actual feature code afterward.

## Lesson 4.4 DOM basics

The Document Object Model is the browser's object representation of the page. document.querySelector finds the first matching element using a CSS-style selector. It returns null if nothing matches. classList changes classes, textContent changes plain text and setAttribute updates an attribute.

```
const button = document.querySelector("#theme-button");
console.log(button);
```

Run this with the complete portfolio HTML from Module 1. You should see a button element, not null. A leading # selects an ID; a leading dot selects a class. The spelling must match exactly. The defer attribute in the previous lesson prevents this selection from running before the HTML is parsed.

### Check

Temporarily use button.textContent = "Theme" and refresh. The selected button is hidden until the feature is ready, but inspecting it in developer tools should show the changed text. Restore “Dark mode” afterward. If the console says it cannot read a property of null, check the selector and the HTML before changing your event code.

## Lesson 4.5 Events

addEventListener connects an event to a function. The function runs later, when the event happens. An arrow function is a compact way to supply that function. Do not call the handler while adding it unless you mean to use its return value.

Temporary experiment: reveal the existing theme button and log each click. Remove this experiment before installing the real feature so one click does not run duplicate handlers.

```
const button = document.querySelector("#theme-button");
button.hidden = false;
button.addEventListener("click", () => {
  console.log("Button clicked!");
});
```

### Check

Click the button twice and confirm two messages. Reach it with Tab and activate it with Enter or Space; a native button supports this automatically. The visible page has not changed yet because the handler only writes to the console. Next, the handler will update a class and accessibility state.

## Lesson 4.6 Build dark and light mode

Add the body.dark variable overrides from Appendix B if you have not already used the complete stylesheet. First implement this minimum version in a clean script.js. It uses the button labelled “Dark mode”; pressed means that dark mode is active.

```
const themeButton = document.querySelector("#theme-button");
themeButton.hidden = false;
themeButton.addEventListener("click", () => {
  const isDark = !document.body.classList.contains("dark");
  document.body.classList.toggle("dark", isDark);
  themeButton.setAttribute("aria-pressed", String(isDark));
});
```

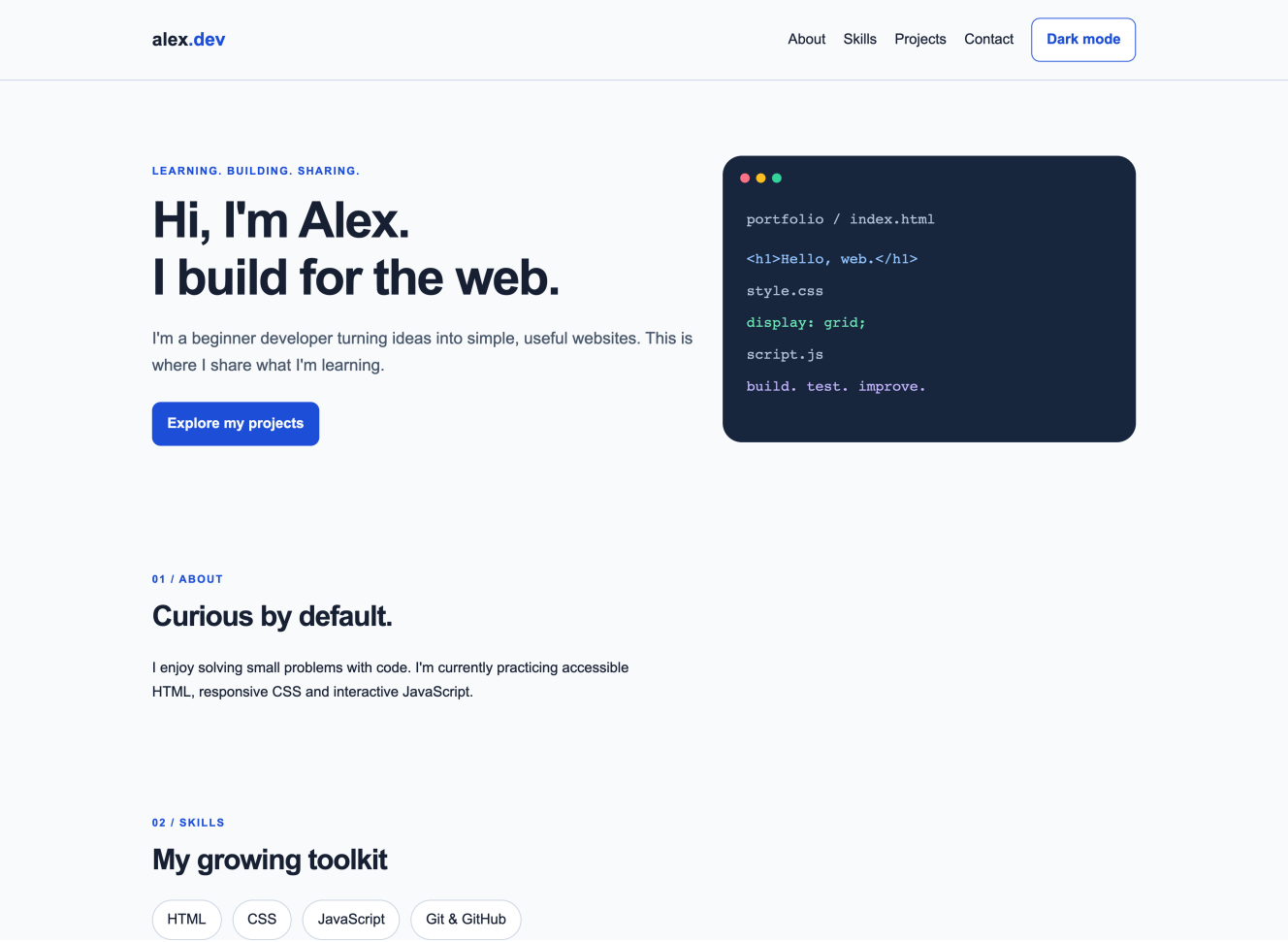
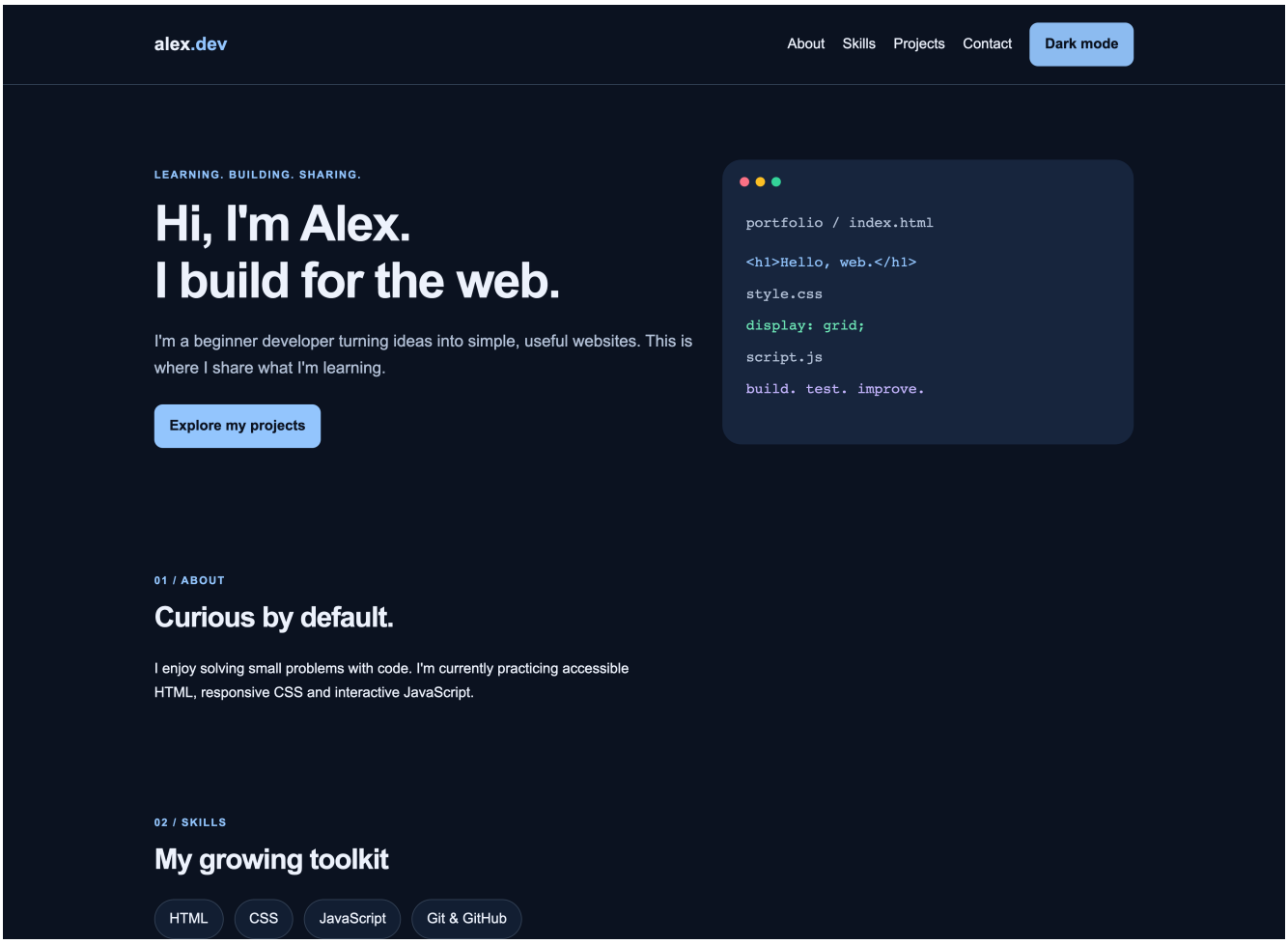



Figure 8. Identical content in light and dark themes. Only the palette and theme state change. Light theme.




Dark theme. The same components retain their layout and accessible controls.

The ! operator reverses a boolean. classList.toggle(name, true) adds a class and false removes it. aria-pressed tells assistive technology the current state; styling alone does not do that. String() converts the boolean to the attribute text “true” or “false”.

### Add persistence

The complete version in Appendix C reads a saved theme from localStorage, otherwise uses prefers-color-scheme. setTheme keeps the class and pressed state together. A click saves the new choice. Storage access is wrapped in try/catch because some browser settings can prevent it. If storage fails, switching still works for the current visit. This version checks the system preference at page load; a live operating-system theme change takes effect on the next load unless the visitor has saved a choice.

Replace the entire minimum script with Appendix C before the next lab. Do not paste both versions together: duplicate const declarations are a syntax error.

## Project 4 Interactive portfolio lab

This project completes the original three interaction requirements: theme toggle, interactive mobile navigation and one additional interaction. The extra feature is project filtering. Read the full script in Appendix C in three blocks, and compare with 05-javascript-complete.

### Lab A Mobile navigation

The HTML button uses aria-controls="nav-links" and aria-expanded="false". The script selects the button and link wrapper. setMenu changes .is-open and aria-expanded together. syncMenu closes the menu and reveals the button only below 640px. A media-query change listener keeps the layout synchronized when the viewport crosses that boundary.

A click toggles the menu. Selecting a navigation link closes it on small screens. Escape closes an open menu and returns focus to its button. The code adds .js to html only after the listeners exist. If scripts are disabled, the links stay visible.

### Lab B Project filters

The HTML buttons have data-filter values all, web and javascript. Each project article has a matching data-category. dataset reads these custom attributes. querySelectorAll returns a collection; the spread syntax [...] converts it to an array. forEach runs a function for each item. The code hides nonmatching cards, updates aria-pressed on every filter and announces the visible count through role="status".

```
// Core decision inside filterProjects(category)
card.hidden = category !== "all" &&
  card.dataset.category !== category;
```

!== means “not equal”, and && means both conditions must be true. With all selected, the first condition is false so no card is hidden. The CSS rule [hidden] { display: none !important; } ensures component layout rules do not accidentally reveal hidden cards.

### Test and pass

- Click the theme button twice; the palette and pressed state change together.
- Choose a theme, refresh on the same origin, and confirm it remains selected when storage is available.
- At 390px, open Menu, choose About, reopen, and press Escape. The menu closes each time; Escape returns focus to Menu.
- Resize from mobile to desktop and back. The navigation remains usable and the mobile menu resets closed.
- Select Web: two cards show. Select JavaScript: one shows. Select All: three show.
- Disable JavaScript and refresh: all navigation links and cards remain visible.

Pass when all checks succeed and the console has no unexpected errors. If a control is absent, confirm the full script is connected and has no earlier error. Never leave the .js class manually added in your HTML.

# Module 5 Git and GitHub

Goal: create a useful project history and share a clear repository. Guided time: about 20 minutes. Continue in your working folder, not in the course bundle's root.

## Lesson 5.1 Why developers use Git

Git records snapshots of selected changes in a repository. A commit gives a snapshot a message and identity. History helps you understand when and why something changed. Git runs locally. GitHub hosts repositories online and adds collaboration tools; it is a separate service.

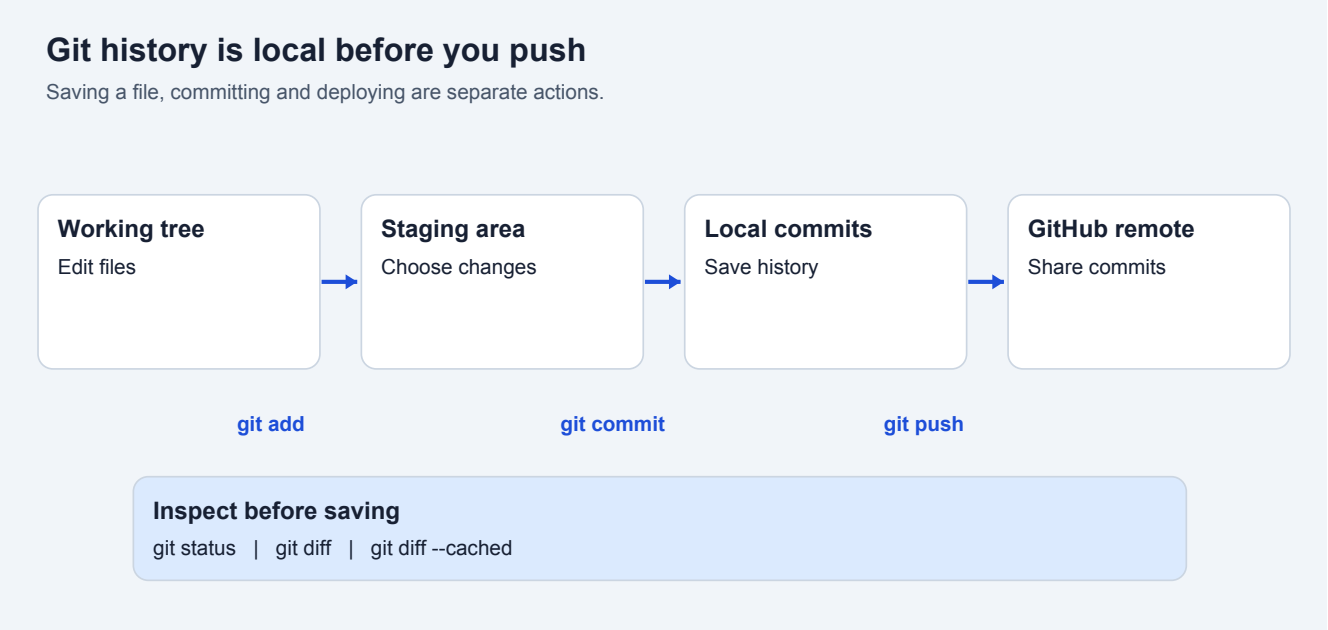



Figure 9. Choose and commit changes locally before sharing them with a remote.

The working tree contains the files you edit. The staging area holds the version of each change you choose for the next commit. A commit saves those staged changes locally. Push sends commits to a remote repository. Saving a file is not committing it, and committing is not publishing the live website.

### Practice

Describe this sequence in your own words: edit → inspect → stage → commit → push. If you edit a file again after staging it, stage the newer change again if you want it included. git status helps distinguish staged and unstaged changes.

## Lesson 5.2 Create a Git repository

Open the VS Code terminal in devfoundry-portfolio. Confirm the folder shown in the terminal is your project before running commands. Configure your name and commit email once. Replace the example values; for privacy, you can use the exact GitHub-provided noreply address from your account email settings.

```
git config --global user.name "Your Name"
git config --global user.email "YOUR_COMMIT_EMAIL"
git init
git branch -M main
git status
```

Create .gitignore in the project root with the following contents. The leading dot is part of the filename. It excludes common local clutter, not files already tracked by Git.

```
.DS_Store
Thumbs.db
.vscode/
```

Inspect your changes before staging. git diff shows unstaged text edits; new untracked files still need to be opened and reviewed. Stage the project, inspect the staged summary and commit.

```
git add .
git diff --cached --stat
git commit -m "Build responsive interactive portfolio"
git status
git log --oneline -3
```

### Check

You should see one commit and a clean working tree. If Git says “Author identity unknown”, set the name and email and retry the commit. If it says “nothing to commit”, you may already be up to date or in the wrong folder. Do not publish passwords, tokens or private contact data: a public portfolio repository is visible to others.

## Lesson 5.3 Upload your project to GitHub

On GitHub, create a repository named devfoundry-portfolio. Use public visibility for this course's free GitHub Pages workflow. Because you already have a local repository, leave README, .gitignore and license initialization off so the remote starts empty. Copy the HTTPS URL GitHub shows.

In the commands below, replace YOUR_USERNAME with your account name. Run them in your local project folder. Do not include placeholder angle brackets in a real command.

```
git remote add origin https://github.com/YOUR_USERNAME/devfoundry-portfolio.git
git remote -v
git push -u origin main
```

Authenticate when prompted using GitHub's supported HTTPS credential flow. Your normal account password is not the password for Git HTTPS operations. Git Credential Manager or GitHub CLI can provide a browser sign-in flow; see the official setup reference. Never put a token into a committed file or share it in a screenshot.

### Check and recover

Refresh the repository page. index.html, style.css, script.js and assets should appear at its root. If origin already exists, inspect git remote -v; correct a wrong URL with git remote set-url origin followed by the correct URL. If the push is rejected because the remote has a README commit, do not force-push. Use an empty new repository for this first exercise or ask for help reconciling the histories.

# Lesson 5.4 Create a professional README

README.md explains what the project does and how someone can inspect it. Markdown uses # for headings, - for lists and [label](URL) for links. Add your actual repository and live website URLs after deployment. A useful README describes the current state honestly, including planned features or projects.

```
# Alex's Portfolio
A responsive personal portfolio built while learning web development.
## Features
- Responsive project grid
- Saved light and dark theme
- Mobile navigation and project filters
## Built with
HTML, CSS and JavaScript. No build step.
## Run locally
Download or clone this repository, then open index.html.
## Live website
Add your GitHub Pages URL after publishing.
```

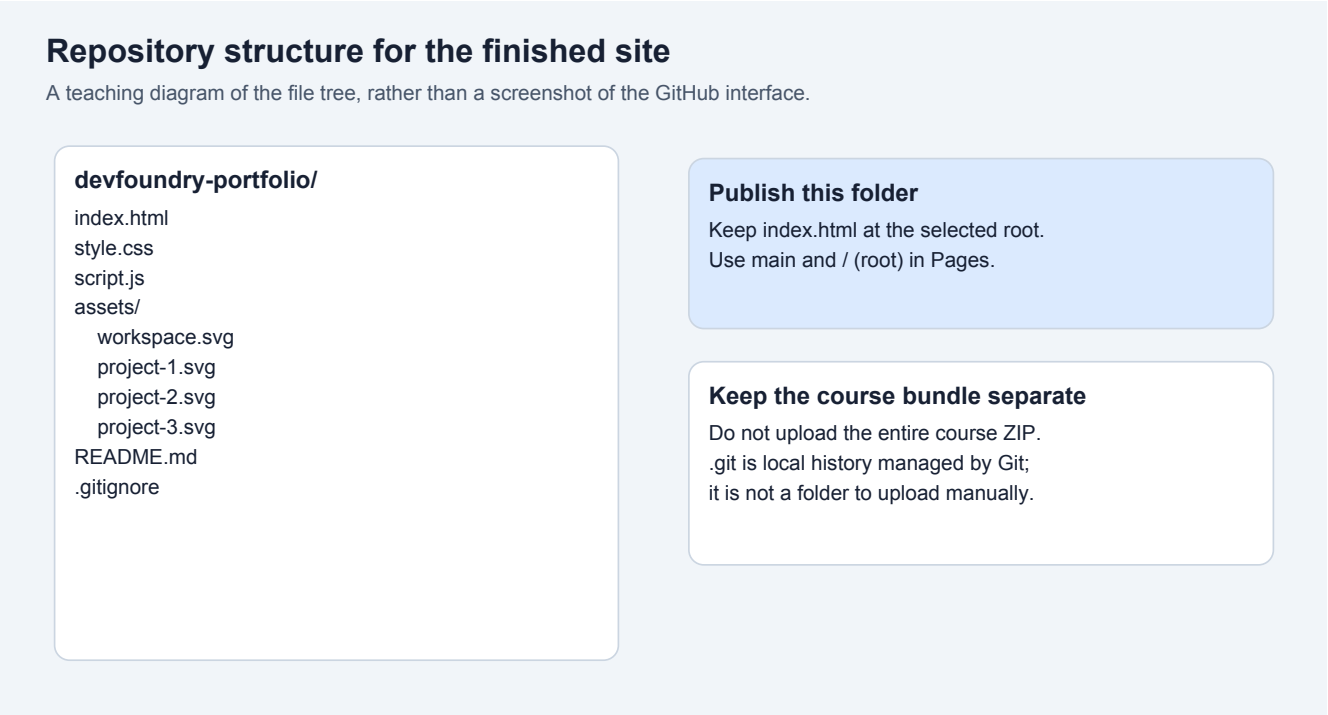



Figure 10. Publish the portfolio folder contents, with index.html at the selected source root.

### Mini project Professional repository

Give the repository a clear description, organized filenames and a README. Add a screenshot if useful: store it at assets/preview.png and reference it with ![Portfolio screenshot](assets/preview.png). The example assets do not include that screenshot; capture your own before adding the Markdown.

Make a second commit for the README and .gitignore if they were not included in the first one. Push and verify the changes appear online. Pass when a new visitor can understand the project, run it locally, identify its main features and find its live URL once Module 6 is complete. Meaningful history explains real changes; do not manufacture dozens of empty commits.

# Module 6 Deploy your website

Goal: publish your static site and verify a real update. Guided time: about 15 minutes, excluding deployment waiting time. GitHub interface labels may change; use the official links in the resource directory when needed.

## Lesson 6.1 What deployment means

Deployment makes your files available from a hosting service. A browser requests a URL, the host serves your files, and the browser renders the page. Git stores history; it does not itself host your website. With this course workflow, a push to the configured GitHub branch starts a Pages build and deployment.

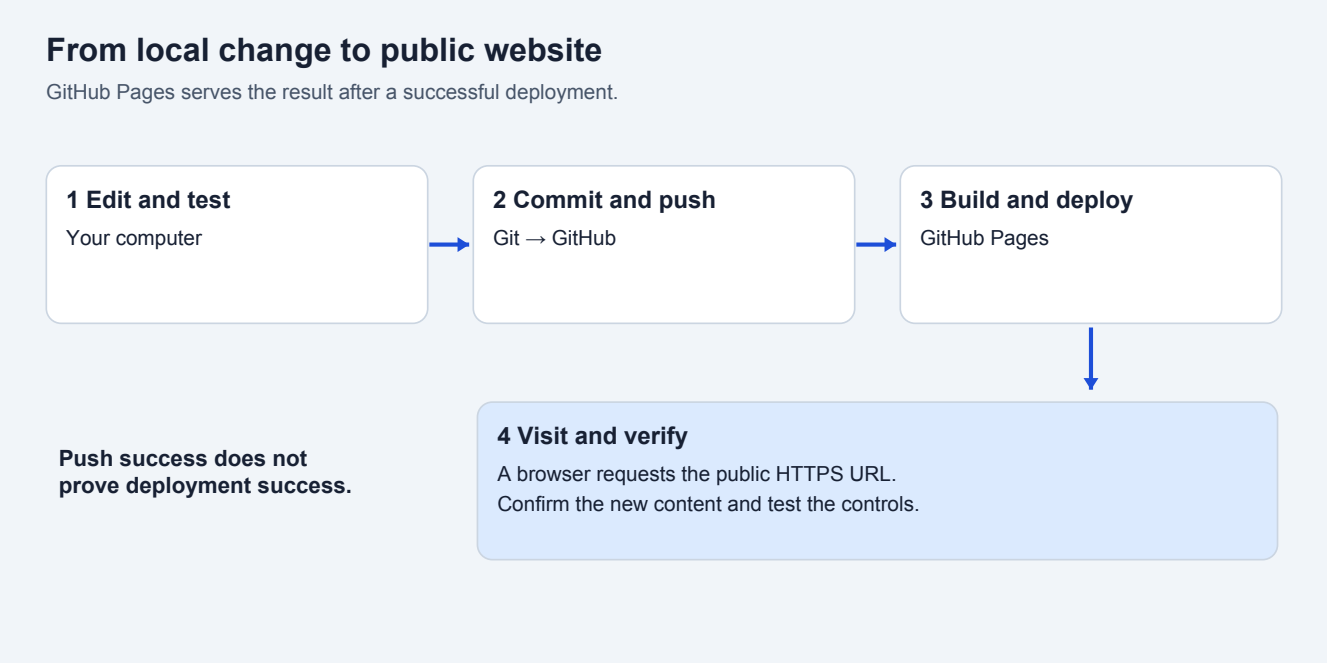



Figure 11. A push starts the publishing workflow; verify the public page after deployment succeeds.

Our site is static HTML, CSS and JavaScript. GitHub Pages can serve it without an application server. It does not run a Python or PHP backend, handle a database or process a contact form for this project. A mailto link delegates contact to the visitor's email app.

### Before publishing

Open the local page and confirm your name, page title, description and contact details are ready to be public. Replace demo email addresses. Remove claims or links that suggest a planned project is already complete. Upload only your portfolio project, not the full paid course ZIP.

## Lesson 6.2 Deploy using GitHub Pages

Use the repository you pushed in Module 5. The steps below use branch deployment and a public repository. You need permission to manage its Pages settings.

1. Open the repository on GitHub. Check that index.html is at the repository root, alongside style.css, script.js and assets.
2. Open Settings, then Pages in the sidebar.
3. Under Build and deployment, choose Deploy from a branch as the source.
4. Choose main and / (root), then Save.
5. Inspect the Pages deployment status or the repository's Actions tab. Wait for a successful build, then use the published site link shown in Pages.
6. Open that URL in a new tab. Test the images, navigation, theme and filters on the live site.

A project site normally has a URL shaped like https://YOUR_USERNAME.github.io/devfoundry-portfolio/. Use the URL GitHub displays rather than guessing it. A repository named YOUR_USERNAME.github.io is a different user-site convention and is not required here. A custom domain is optional.

### Check and troubleshoot

A successful push is not proof of successful deployment. If the site is missing, check branch and source folder, the exact lowercase index.html filename and the latest deployment result. If styles or images are missing, check relative paths and case. Use assets/project-1.svg instead of /assets/project-1.svg: a leading slash starts at the domain root and can break a project site under a repository path.

Official instructions: GitHub Docs, “Creating a GitHub Pages site” and “Configuring a publishing source”. These support the branch workflow above; links appear in the resource directory.

## Lesson 6.3 Update your live website

Change one visible sentence in your local About section. Save and test locally. Inspect the change, commit it and push it. Your first push in Module 5 set the upstream, so git push can now use it.

```
git diff
git add index.html
git diff --cached
git commit -m "Clarify the About section"
git push
```

After the deployment completes, refresh the public URL and find the new sentence. If it still looks old, confirm the remote commit contains your change and the Pages job succeeded. Then try a hard refresh or a private browsing window. Repeatedly refreshing before a deployment finishes will not speed it up.

### Check

Record the live URL, the commit message and the changed sentence in challenges/test-log.md. This proves that you understand the complete loop: edit locally, test, commit, push, deploy and verify. Add the working live URL to README.md in another normal commit.

# Lesson 6.4 Domains

Hosting serves the website. A domain is a human-readable name that can point toward that hosting. DNS records connect names to destinations. HTTPS encrypts traffic between a browser and the host. You do not need to buy a domain to finish this course; the GitHub Pages URL is enough.

If you later choose a domain, follow GitHub's current custom-domain instructions and your registrar's directions. DNS records differ for an apex domain and a subdomain; use the settings for your own configuration. Verify the domain and HTTPS state afterward.

# Module 7 Final project

## Build your developer portfolio

Your final task is to personalize and explain the website independently. You may refer to this workbook and the complete solution, but make meaningful decisions about content, design and behavior. The goal is a site you can understand and maintain, not a screenshot you copied without understanding.

### Required content and features

- Navigation that reaches all major sections, with a usable mobile menu.
- Hero with your name, focus and a clear next action.
- About section and a genuine skills list.
- At least three project cards with titles, descriptions and images or meaningful visual alternatives. Clearly identify unfinished concepts.
- Contact section with a working public contact method, plus a footer.
- Responsive layout from approximately 320px through desktop, with no horizontal page overflow.
- Consistent CSS styling, readable contrast, visible keyboard focus, and hover or transition feedback with reduced-motion support.
- Working dark/light toggle and project filters, alongside the mobile-menu interaction from Project 4.
- Git repository, useful README, meaningful commits, public deployment and one verified live update.

### Personalization brief

Replace all example identity details and metadata. Write your own introduction and project descriptions. Change at least two visual choices, such as the accent palette and card styling, while preserving readability. Add at least one real project destination when you have work available; do not use # links to imply an external project exists. Explain your three interactions in a short README section.

### Suggested independent build sequence

Sketch the sections on paper. Write your content before choosing colors. Rebuild or review the HTML landmarks. Apply the shared design tokens, then layout. Test the narrow viewport before adding more motion. Implement one interaction at a time and test it. Review your repository, deploy, and check the live URL in a fresh browser tab.

### Bonus challenges

Save the theme preference if you have only implemented the minimum toggle. Add a custom favicon. Build another actual project and replace a planned concept. Add a gentle scroll animation with reduced-motion support. Extend the project filters with another category and a zero-result message. Connect a custom domain later.

A contact form is an advanced extension: static HTML alone does not deliver messages. Use a service or backend, provide labels and status feedback, and avoid claiming submission succeeded until the service confirms it. Do not place private service credentials in frontend JavaScript.

## Final project assessment

Use this rubric to assess work consistently. Score each area from 0 to 4: 0 absent, 1 substantially incomplete, 2 works with important gaps, 3 meets the requirement, 4 meets it and you can explain your choices and testing. This is a self-assessment tool, not an accredited certification.

| Area | Evidence for a score of 3 |
| --- | --- |
| Content and HTML | Required sections, honest project descriptions, meaningful headings and unique IDs |
| CSS and visual clarity | Reusable styling, readable text, consistent cards, no distorted images |
| Responsive behavior | 320px, 390px, 768px and 1440px checks pass without horizontal overflow |
| JavaScript and usability | Theme, menu and filters work; keyboard and no-script fallbacks checked |
| Repository and delivery | README, meaningful commits, live URL and verified update recorded |

Target: at least 15 out of 20, with every required item complete. A high total does not override a broken deployment or inaccessible navigation. Fix missing essentials and rerun the checks.

### Record your assessment

Content and HTML: ____ / 4 CSS and clarity: ____ / 4

Responsive behavior: ____ / 4 JavaScript and usability: ____ / 4

Repository and delivery: ____ / 4 Total: ____ / 20

Which requirement needs the most work? Write the issue, the change you plan to make and the check that will prove it is fixed.

What can you now explain without looking at the reference solution? Choose one HTML decision, one CSS rule and one JavaScript interaction.

# Final project checklist

[ ] My name, page title, metadata and contact details are correct.

[ ] Navigation, hero, About, Skills, Projects, Contact and footer exist.

[ ] At least three project cards have honest descriptions and meaningful visuals.

[ ] Finished links reach real destinations; unfinished work is clearly labelled.

[ ] Images use correct relative paths, dimensions and alt text.

[ ] There is no page-level horizontal scrolling at the four test widths.

[ ] Text remains available at 200% zoom; keyboard focus is visible.

[ ] The mobile menu opens, closes, handles Escape and survives resizing.

[ ] Theme state and its accessibility attribute agree; saved choice restores when available.

[ ] Filtering shows two Web cards, one JavaScript card and all three on reset, or the matching counts for my own categories.

[ ] All links and project cards remain available without JavaScript.

[ ] Reduced-motion settings suppress unnecessary card movement.

[ ] The console has no unexpected errors on the local or deployed page.

[ ] My repository contains source files, assets, .gitignore and README.

[ ] My commit history describes real changes and contains no private credentials.

[ ] My live URL loads and I have verified a content update after deployment.

### Submission package

Prepare the live website URL, GitHub repository URL, one desktop screenshot, one mobile screenshot and a short paragraph explaining what you changed, what was difficult and what you would improve next. Save the completed checklist and test log with your own learning notes. You have finished when another person can open the site and follow its main paths without your help.

# Module 8 Where to go next

You have now practiced the whole path from files to a public website. The next step is repetition with a slightly different problem. Build a landing page for a fictional service or a small information site, then deploy it using the same workflow. Changing the content and constraints will reveal which ideas you understand independently.

## A practical learning path

1. Strengthen HTML and CSS: forms, accessibility, more layout practice and browser debugging.
2. Learn JavaScript more deeply: arrays, objects, functions, control flow, modules, errors and asynchronous code.
3. Use APIs: fetch data, handle loading and failure states, and understand what must remain on a server.
4. Explore React after you can build and debug ordinary JavaScript interactions.
5. Learn backend development, databases, authentication and full-stack deployment when your project needs them.

A possible next DevFoundry course is “JavaScript From Beginner to Building Real Apps.” Treat that as a learning direction; this workbook does not promise an existing follow-on product, release date or included access.

## Your next seven practice sessions

Session 1: improve your About content. Session 2: add one real project. Session 3: test keyboard and zoom behavior. Session 4: write a small JavaScript widget without copying. Session 5: improve the README. Session 6: deploy a visible update and check it on a phone. Session 7: write down three things you can now explain and one topic to study next.

## Explain your website

Before calling the course complete, answer these questions without reading the code: Where does the HTML describe the project cards? Which CSS rule sets the number of columns? What happens when the theme button is clicked? What is staged before a commit? Which branch is Pages publishing? If an answer is unclear, revisit that lesson and trace one example.

# Course downloads and quick references

The ZIP is an actual companion to this PDF. Its checkpoints contain complete files, not just empty folder labels. Copy the starter into your own working folder and use later snapshots to diagnose differences.

```
DevFoundry-Downloads/
  01-starter-files/
  02-html-complete/
  03-css-complete/
  04-responsive-complete/
  05-javascript-complete/
  06-final-project/
  challenges/
  cheatsheets/
  visuals/
  README.md
```

Each checkpoint includes index.html, style.css, script.js, a README and bundled assets. Inactive CSS or JavaScript files contain only a comment until their module begins. The HTML checkpoint is unstyled. The CSS checkpoint has the deliberate fixed desktop width. The responsive checkpoint keeps links visible without scripts. The JavaScript and final checkpoints include all three interactions.

challenges contains practice briefs, an answer guide, a final checklist and a test log. cheatsheets contains the three references on the following pages. visuals contains the instructional diagrams and reference screenshots. All code used by the finished portfolio is also printed in the appendices.

## Working with PDF code

Use the editable source files when possible. PDF viewers sometimes alter indentation or wrap long lines during copy and paste. Do not copy line labels, page footers or prose into a source file. When pasting code, preserve quotes, braces, punctuation and backticks. Review your browser console after changes.

# HTML cheat sheet

| Element or attribute | Use |
| --- | --- |
| <!DOCTYPE html> | Request modern HTML behavior |
| html lang="en" | Declare the page language |
| head / title / meta | Metadata, tab title and viewport setup |
| header / nav / main / footer | Meaningful page regions |
| section / article / div | Named topic, independent item, neutral wrapper |
| h1 / h2 / h3 / p | Content hierarchy and paragraphs |
| ul / li | Unordered list and list item |
| a href="#about" | Navigate to the unique id="about" |
| img src="..." alt="..." | Image path and text alternative |
| button | A native keyboard-accessible action control |
| class / id | Reusable grouping / unique identifier |
| hidden / data-category | Hide inactive content / store a custom value |

Checklist: valid nesting, matching closing tags, one main landmark, useful text, working paths and unique IDs. Use a button for an action and a link for a destination.

# CSS cheat sheet

| Pattern | Meaning |
| --- | --- |
| .card / #projects / h1 | Class / ID / element selector |
| color / background | Text color / background fill |
| var(--accent) | Read a custom property |
| padding / margin / gap | Inner space / outer space / space between items |
| border / border-radius | Edge / corner rounding |
| box-sizing: border-box | Include padding and border in a declared width |
| display: flex | Arrange direct children on a main axis |
| justify-content / align-items | Align on main axis / cross axis |
| flex-wrap: wrap | Allow flex items onto another line |
| display: grid | Create a row and column layout |
| repeat(3, minmax(0, 1fr)) | Three equally flexible Grid tracks |
| width: min(1100px, calc(100% - 2rem)) | Bounded fluid page width |
| max-width: 100%; height: auto | Let images shrink proportionally |
| @media (min-width: 640px) | Apply rules at or above this viewport width |
| :hover / :focus-visible | Pointer feedback / visible keyboard focus |
| transition / transform | Smooth a change / visually move an element |

When styling fails, check the matching selector, stylesheet path, cascade, spelling and missing braces. Inspect the element instead of randomly increasing selector specificity. The final course's only !important rule protects the native hidden attribute from layout overrides.

# Git cheat sheet

| Command | What it does |
| --- | --- |
| git --version | Check that Git is installed |
| git init | Create a repository in the current folder |
| git status | Show staged, unstaged and untracked changes |
| git diff | Inspect unstaged text changes |
| git add index.html | Stage one file's current changes |
| git add . | Stage changes under the current directory |
| git diff --cached | Inspect the staged changes |
| git commit -m "Message" | Save the staged snapshot locally |
| git log --oneline -5 | Review the five latest commits |
| git remote -v | Inspect remote repository addresses |
| git push -u origin main | First push and upstream setup |
| git push | Send new commits to the configured upstream |
| git pull --ff-only | Get remote changes only if a fast-forward is possible |

If pull refuses because histories diverged, stop and inspect rather than using force or discarding work. Pull is useful when the remote changed on another computer or in the web editor. It is not a substitute for committing your own local work. git status and reading the error are your first recovery tools.

# Challenges and answer guide

Try each task before reading the answer. Your solution can look different if it satisfies the behavior and remains readable.

## Challenge 1 Content and navigation

Add a “Now learning” section with a heading and paragraph. Link to it from navigation. Pass when the new link moves to that section and its ID is unique.

Answer: create <section id="learning"> with an h2 and p inside main; add <a href="#learning">Now learning</a> inside #nav-links. Style it using the existing container and section classes. The menu's link-click handler already handles this new anchor because it listens on the wrapper.

## Challenge 2 Card spacing

Increase the inside space in every card while preserving equal column widths. Pass when cards fit the same grid without overflow.

Answer: change .card-body padding from 1.4rem to 1.75rem. Do not add a fixed card width. border-box keeps declared sizing predictable, while Grid controls track widths.

## Challenge 3 Responsive layout

Make the tablet layout begin at 700px instead of 640px. What else would you need to change if you wanted the navigation to use the same breakpoint?

Answer: change the project Grid min-width query to 700px. The navigation breakpoint is independent. To move it too, change both the CSS max-width: 639px menu query and the JavaScript matchMedia query to 699px. Keep related conditions synchronized and retest boundary widths.

## Challenge 4 Interaction

Add an “Other” filter with a matching fourth card. Pass when it shows only matching cards and the status count is correct.

Answer: add a button with data-filter="other" and aria-pressed="false" inside #filters, then an article with data-category="other" inside .projects-grid. No JavaScript logic changes are required if the DOM elements exist before the deferred script runs. The function reads categories rather than hard-coding button names. If there are zero matches, the existing count reports “0 projects shown”; a richer empty-state message is an optional improvement.

## Challenge 5 Delivery

Make one small content improvement, commit it and prove the public site updated.

Answer: edit, save, inspect with git diff, stage the changed file, inspect git diff --cached, commit with a clear message, push and wait for a successful deployment. Record the changed sentence and live URL. A repository screenshot alone does not prove the live page changed.

# Troubleshooting desk

## My HTML looks wrong or content disappears

Check nesting and closing tags around the first broken region. Make sure you did not paste Markdown fences into the HTML file. Inspect the browser DOM to see how it interpreted your markup. Compare one section with the checkpoint rather than replacing unrelated working sections.

## My stylesheet does not load

Confirm the file is style.css, the link is inside head, and the href is relative to index.html. Save both files. Inspect the failed request in developer tools. A missing brace can affect rules below it, so check the first malformed block before adding another rule.

## An image works locally but fails online

Match the exact filename and capitalization. Verify it was committed and appears on GitHub. Avoid computer-specific file paths and leading-slash asset URLs for this project site. Remember that the public site may be case-sensitive even when your local filesystem is forgiving.

## The theme button or menu never appears

Hidden controls are revealed by JavaScript. Look for the first console error, verify the script path and defer attribute, and compare selectors with IDs. If you pasted both the experimental and final scripts, duplicate const declarations can prevent everything from running. Replace the whole file with the single complete implementation.

## Theme persistence behaves differently locally

Storage rules for file:// URLs vary by browser. Use a local preview server or the deployed HTTPS site to verify persistence on a stable origin. localStorage is specific to an origin and browser profile; a preference saved on your local preview does not automatically transfer to your public website.

## Git rejects my push

Read the error. Check git status, git remote -v and your authenticated account. A permission error differs from a history conflict. Do not resolve an unfamiliar rejection with a force-push. For this course, starting with an empty remote avoids the common separate-README-history problem.

## Pages is blank or returns 404

Check that the selected branch exists and contains index.html at the selected root. Confirm the deployment succeeded and use the URL shown in Pages. A GitHub repository page is not the same URL as the live website. If the host reports an error, inspect the latest workflow log before editing random CSS.

# Resource directory

The explanations and diagrams in this course are DevFoundry teaching material. Use these official references for deeper study and for setup or hosting screens that may change. Service instructions were checked in November 2026.

## Tools and setup

Visual Studio Code getting started: https://code.visualstudio.com/docs/getstarted/overview

Git downloads: https://git-scm.com/downloads

GitHub Git setup and authentication: https://docs.github.com/en/get-started/git-basics/set-up-git

## Web platform reference

MDN learning hub: https://developer.mozilla.org/en-US/docs/Learn_web_development

MDN responsive design: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design

MDN HTML reference: https://developer.mozilla.org/en-US/docs/Web/HTML

MDN CSS reference: https://developer.mozilla.org/en-US/docs/Web/CSS

MDN JavaScript reference: https://developer.mozilla.org/en-US/docs/Web/JavaScript

## GitHub Pages

Create a Pages site: https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site

Configure the branch and source folder: https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site

Custom domains: https://docs.github.com/en/pages/configuring-a-custom-domain-for-your-github-pages-site

## Visual reference index

The following figures are original diagrams or screenshots of the supplied portfolio: final portfolio target, browser file roles, HTML landmarks, CSS box model, Flexbox axes, Grid tracks, desktop/tablet/mobile layouts, light/dark states, Git workflow, repository tree and deployment flow. The repository tree is a teaching diagram, not a claim to show GitHub's current interface. The screenshots use the actual course source and bundled illustrations.

# Appendix A Complete portfolio HTML

Use this complete file as a reference after Module 4. For the unstyled HTML checkpoint, leave out the stylesheet and script lines. For the CSS and responsive checkpoints, include the stylesheet but leave out the script line. The download snapshots already apply these differences.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Alex Morgan's web development portfolio.">
  <title>Alex Morgan | Web Developer</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <a class="skip-link" href="#main">Skip to content</a>
  <header class="site-header">
    <nav class="container nav" aria-label="Main navigation">
      <a class="brand" href="#home">alex<span>.dev</span></a>
      <button id="menu-button" class="menu-button" hidden
        aria-controls="nav-links" aria-expanded="false">Menu</button>
      <div id="nav-links" class="nav-links">
        <a href="#about">About</a>
        <a href="#skills">Skills</a>
        <a href="#projects">Projects</a>
        <a href="#contact">Contact</a>
      </div>
      <button id="theme-button" hidden aria-pressed="false">Dark mode</button>
    </nav>
  </header>
  <main id="main">
    <section id="home" class="hero container">
      <div>
        <p class="eyebrow">LEARNING. BUILDING. SHARING.</p>
        <h1>Hi, I'm Alex.<br>I build for the web.</h1>
        <p class="intro">I'm a beginner developer turning ideas into simple,
          useful websites. This is where I share what I'm learning.</p>
        <a class="button" href="#projects">Explore my projects</a>
      </div>
      <img class="hero-art" src="assets/workspace.svg" width="520"
        height="360"
    alt="Illustrated code editor with HTML, CSS and JavaScript files">
    </section>
    <section id="about" class="container section">
      <p class="eyebrow">01 / ABOUT</p>
      <h2>Curious by default.</h2>
      <p>I enjoy solving small problems with code. I'm currently practicing
        accessible HTML, responsive CSS and interactive JavaScript.</p>
    </section>
    <section id="skills" class="container section">
      <p class="eyebrow">02 / SKILLS</p>
      <h2>My growing toolkit</h2>
      <ul class="skills">
```

```
      <li>HTML</li><li>CSS</li><li>JavaScript</li><li>Git &amp; GitHub</li>
    </ul>
  </section>
  <section id="projects" class="container section">
    <p class="eyebrow">03 / PROJECTS</p>
    <h2>Small projects. Real progress.</h2>
    <div id="filters" class="filters" hidden
      role="group" aria-label="Filter projects">
      <button data-filter="all" aria-pressed="true">All</button>
      <button data-filter="web" aria-pressed="false">Web</button>
      <button data-filter="javascript" aria-pressed="false">JavaScript</button>
    </div>
    <p id="project-count" class="muted" role="status" hidden></p>
    <div class="projects-grid">
      <article class="project-card" data-category="web">
        <img src="assets/project-1.svg" width="600" height="320"
          alt="Portfolio concept with a hero section and three project cards"
  loading="lazy">
        <div class="card-body">
          <p class="eyebrow">HTML / CSS</p>
          <h3>Personal portfolio</h3>
          <p>A home for my projects, skills and learning journey.</p>
          <a href="#home">View this portfolio</a>
        </div>
      </article>
      <article class="project-card" data-category="web">
        <img src="assets/project-2.svg" width="600" height="320"
          alt="Practice landing page concept with a headline and call to action"
  loading="lazy">
        <div class="card-body">
          <p class="eyebrow">PRACTICE CONCEPT</p>
          <h3>Product landing page</h3>
          <p>A planned practice project for layout and typography.</p>
          <span class="muted">Coming soon</span>
        </div>
      </article>
      <article class="project-card" data-category="javascript">
        <img src="assets/project-3.svg" width="600" height="320"
          alt="Planned task list app concept showing three example tasks"
  loading="lazy">
        <div class="card-body">
          <p class="eyebrow">PRACTICE CONCEPT</p>
          <h3>Task list app</h3>
          <p>A planned JavaScript project for practicing DOM events.</p>
          <span class="muted">Coming soon</span>
        </div>
      </article>
    </div>
  </section>
  <section id="contact" class="container section contact">
    <p class="eyebrow">04 / CONTACT</p>
    <h2>Let's build something.</h2>
    <p>Want to share feedback or talk about a project?</p>
    <a class="button" href="mailto:alex@example.com">Email Alex</a>
    <p
  class="muted">Demo address. Replace it with your own public contact
  address.</p>
```

```
    </section>
  </main>
  <footer class="container footer">
    <p>Alex Morgan. Built with HTML, CSS and JavaScript.</p>
    <a href="#home">Back to top</a>
  </footer>
</body>
</html>
```

# Appendix B Complete portfolio CSS

The base desktop design ends immediately before the comment beginning “3.2 Mobile first overrides and breakpoints”. Module 2 uses the base only. Module 3 appends the rest. The dark variables can exist before the JavaScript toggle; the class simply will not be switched yet.

```
/* 2.2 Theme and type */
:root {
  color-scheme: light;
  --bg: #f8fafc;
  --surface: #ffffff;
  --text: #172033;
  --muted: #475569;
  --accent: #1d4ed8;
  --line: #cbd5e1;
}
body.dark {
  color-scheme: dark;
  --bg: #0b1220;
  --surface: #121e32;
  --text: #edf2fc;
  --muted: #b4c2d8;
  --accent: #93c5fd;
  --line: #334155;
}
/* 2.3 Box model and shared elements */
* { box-sizing: border-box; }
body {
  margin: 0;
  background: var(--bg);
  color: var(--text);
  font-family: Arial, Helvetica, sans-serif;
  font-size: 1rem;
  line-height: 1.65;
}
[hidden] { display: none !important; }
a { color: var(--accent); text-underline-offset: .2em; }
button, .button {
  display: inline-block;
  border: 1px solid var(--accent);
  border-radius: .6rem;
  padding: .65rem 1rem;
  font: inherit;
  font-weight: 700;
  cursor: pointer;
  background: var(--surface);
  color: var(--accent);
}
.button { background: var(--accent); color: var(--bg); text-decoration: none; }
button[aria-pressed="true"] { background: var(--accent); color: var(--bg); }
:focus-visible { outline: 3px solid var(--accent); outline-offset: 4px; }
.container { width: 1100px; margin-inline: auto; }
img { display: block; }
h1, h2, h3 { line-height: 1.15; letter-spacing: -.035em; }
h1 { font-size: 3.5rem; margin-block: .6rem 1.25rem; }
```

```
h2 { font-size: 2rem; margin-top: .5rem; }
h3 { font-size: 1.35rem; }
p { max-width: 65ch; }
.eyebrow { color: var(--accent); font-size: .75rem; font-weight: 700;
    letter-spacing: .12em; }
.muted { color: var(--muted); }
.intro { color: var(--muted); font-size: 1.15rem; margin-bottom: 1.6rem; }
.section { padding-block: 3rem; }
/* 2.4 Flexbox */
.site-header { border-bottom: 1px solid var(--line); }
.nav { display: flex; align-items: center; gap: 1.25rem; padding-block: 1.25rem; }
.brand {
  margin-right: auto;
  font-size: 1.3rem;
  font-weight: 800;
  text-decoration: none;
  color: var(--text);
}
.brand span { color: var(--accent); }
.nav-links { display: flex; gap: 1.25rem; }
.nav-links a { color: var(--text); text-decoration: none; }
.hero { display: flex; align-items: center; gap: 3rem; padding-block: 5rem; }
.hero > div { flex: 1; }
.hero-art { width: 42%; height: auto; }
.skills, .filters { display: flex; flex-wrap: wrap; gap: .75rem; }
.skills { padding: 0; list-style: none; }
.skills li {
  padding: .5rem 1rem;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 2rem;
}
/* 2.5 Grid */
.projects-grid { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 1.5rem; }
.project-card {
  overflow: hidden;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: .9rem;
}
.project-card img { width: 100%; height: auto; }
.card-body { padding: 1.4rem; }
.card-body p { color: var(--muted); }
/* 2.6 Feedback */
.project-card { transition: transform .2s ease; }
.project-card:hover, .project-card:focus-within { transform: translateY(-4px); }
a:hover { text-decoration: underline; }
button:hover, .button:hover { filter: brightness(.95); }
.contact { border-top: 1px solid var(--line); }
.footer {
  display: flex;
  justify-content: space-between;
  gap: 1rem;
  padding-block: 2rem;
  border-top: 1px solid var(--line);
}
```

```
.skip-link {
  position: absolute;
  left: 1rem;
  top: -5rem;
  padding: .6rem;
  background: var(--surface);
  z-index: 10;
}
.skip-link:focus { top: .5rem; }
/* 3.2 Mobile first overrides and breakpoints */
.container { width: min(1100px, calc(100% - 2rem)); }
img { max-width: 100%; height: auto; }
h1 { font-size: clamp(2.25rem, 5vw, 3.5rem); }
.hero { flex-direction: column; align-items: stretch; gap: 2rem; padding-block:
    3rem; }
.hero-art { width: min(100%, 520px); align-self: center; }
.projects-grid { grid-template-columns: 1fr; }
.nav { flex-wrap: wrap; }
.nav-links { flex-wrap: wrap; }
.footer { flex-direction: column; }
@media (min-width: 640px) {
  .projects-grid { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}
@media (min-width: 960px) {
  .projects-grid { grid-template-columns: repeat(3, minmax(0, 1fr)); }
  .hero { flex-direction: row; align-items: center; padding-block: 5rem; }
  .hero-art { width: 42%; }
  .footer { flex-direction: row; }
}
/* 3.4 Menu enhanced by JavaScript; links work without it */
@media (max-width: 639px) {
  .nav-links { width: 100%; flex-direction: column; order: 3; }
  .nav-links a { padding-block: .5rem; }
  .js .nav-links { display: none; }
  .js .nav-links.is-open { display: flex; }
}
@media (prefers-reduced-motion: reduce) {
  .project-card { transition: none; }
  .project-card:hover, .project-card:focus-within { transform: none; }
}
```

# Appendix C Complete portfolio JavaScript

Replace script.js with this whole file after completing Lesson 4.6 and Project 4. It contains one set of declarations and handlers for the three features. The HTML IDs, class names, data attributes and media-query widths must match the corresponding source files.

```
// 4.6 Theme: use a saved choice, otherwise use the system preference.
const themeButton = document.querySelector("#theme-button");
const systemTheme = window.matchMedia("(prefers-color-scheme: dark)");
let savedTheme = null;
try { savedTheme = localStorage.getItem("theme"); } catch {}
function setTheme(isDark) {
  document.body.classList.toggle("dark", isDark);
  themeButton.setAttribute("aria-pressed", String(isDark));
}
setTheme(savedTheme === "dark" ||
  (savedTheme !== "light" && systemTheme.matches));
themeButton.hidden = false;
themeButton.addEventListener("click", () => {
  const isDark = !document.body.classList.contains("dark");
  setTheme(isDark);
  try { localStorage.setItem("theme", isDark ? "dark" : "light"); } catch {}
});
// Project 4: mobile menu with keyboard and resize support.
const menuButton = document.querySelector("#menu-button");
const navLinks = document.querySelector("#nav-links");
const smallScreen = window.matchMedia("(max-width: 639px)");
function setMenu(isOpen) {
  navLinks.classList.toggle("is-open", isOpen);
  menuButton.setAttribute("aria-expanded", String(isOpen));
}
function syncMenu() {
  setMenu(false);
  menuButton.hidden = !smallScreen.matches;
}
menuButton.addEventListener("click", () => {
  setMenu(menuButton.getAttribute("aria-expanded") !== "true");
});
navLinks.addEventListener("click", (event) => {
  if (event.target.closest("a") && smallScreen.matches) setMenu(false);
});
document.addEventListener("keydown", (event) => {
  if (event.key === "Escape" && smallScreen.matches &&
      menuButton.getAttribute("aria-expanded") === "true") {
    setMenu(false);
    menuButton.focus();
  }
});
smallScreen.addEventListener("change", syncMenu);
document.documentElement.classList.add("js");
syncMenu();
// Project 4: project filtering is the additional interaction.
```

```
const filters = document.querySelector("#filters");
const cards = [...document.querySelectorAll(".project-card")];
const count = document.querySelector("#project-count");
function filterProjects(category) {
  let visible = 0;
  cards.forEach((card) => {
    card.hidden = category !== "all" && card.dataset.category !== category;
    if (!card.hidden) visible += 1;
  });
  filters.querySelectorAll("button").forEach((button) => {
    button.setAttribute("aria-pressed", String(button.dataset.filter ===
    category));
  });
  count.textContent = `${visible} project${visible === 1 ? "" : "s"} shown`;
}
filters.addEventListener("click", (event) => {
  const button = event.target.closest("button[data-filter]");
  if (button) filterProjects(button.dataset.filter);
});
filters.hidden = false;
count.hidden = false;
filterProjects("all");
```